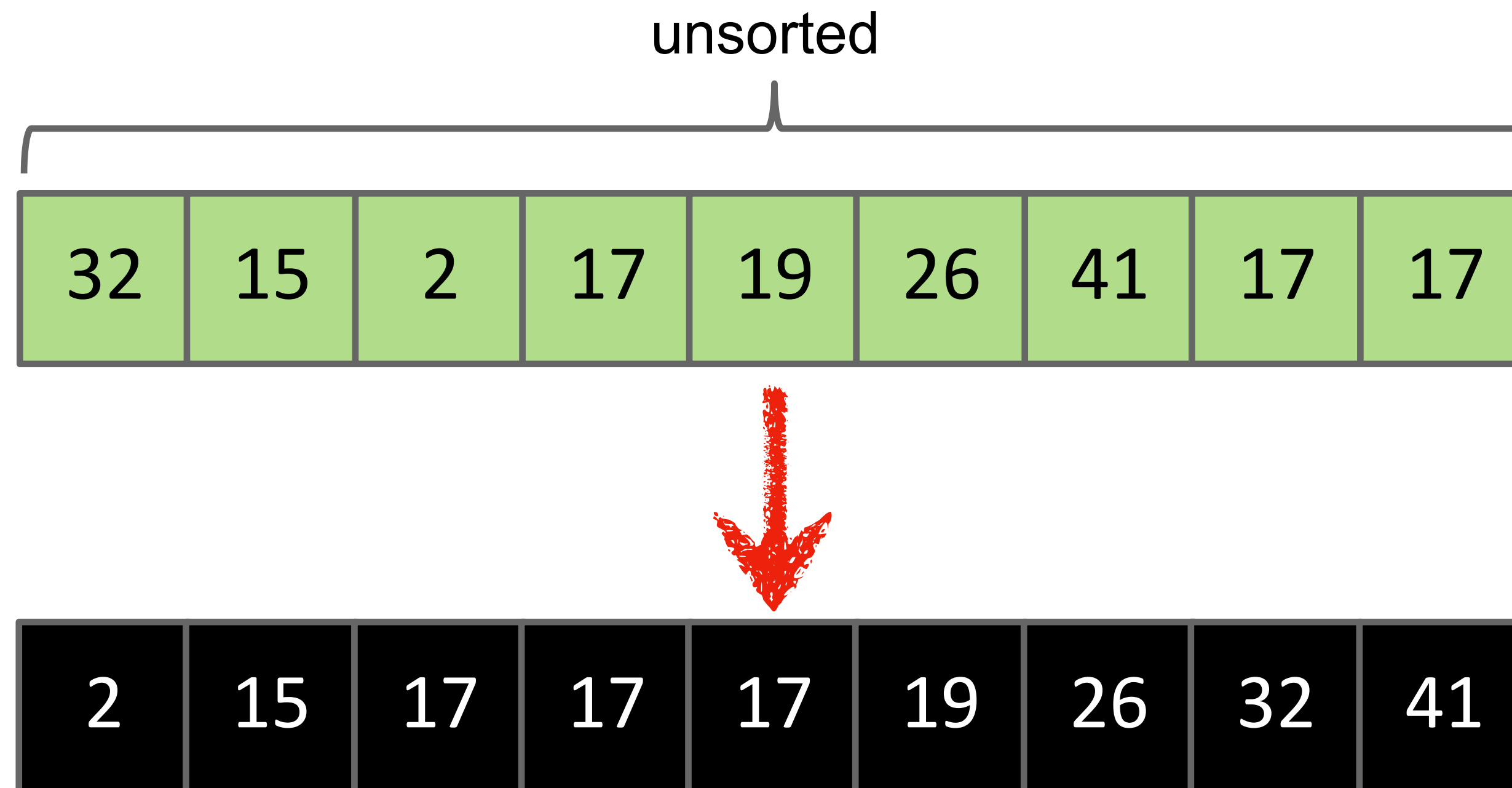


# Lecture 26

## Sorting

# Sorting: unordered → ordered

- Sorting takes an unordered collection and generates an ordered one



# Sorting: unordered → ordered

- There are two main considerations, time complexity and extra storage requirements
- We will cover some of the most important algorithms:
  - Selection sort
  - Insert sort
  - Merge sort
  - Quick sort

# Selection sort

- Find the smallest item within the unsorted part
- Swap the found item above with the first item within the unsorted part
- And mark the found item as 'fixed' so that this item also becomes a part of the sorted part
- Repeat for the unfixed items until all items are 'fixed'

|  |   |
|--|---|
|  | 8 |
|  | 5 |
|  | 2 |
|  | 6 |
|  | 9 |
|  | 3 |
|  | 1 |
|  | 4 |
|  | 0 |
|  | 7 |

# Selection sort - time complexity

- Pseudocode:

```
while the size of the unsorted part is greater than 1:  
    • find the position of the smallest element in the  
      unsorted part  
    • move (swap) this element into the last position of  
      the sorted part  
    • decrement the size of the unsorted part by 1.
```

**Class[Buzz]: selection\_sort**

|   | Worst    | Best        |
|---|----------|-------------|
| A | $O(n)$   | $O(n)$      |
| B | $O(n)$   | $O(\log n)$ |
| C | $O(n^2)$ | $O(n)$      |
| D | $O(n^2)$ | $O(n^2)$    |

# Insertion sort

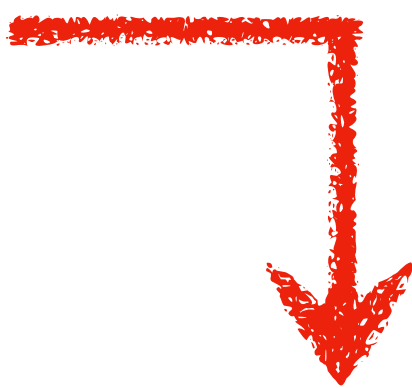
- Move (and fix) one item at a time to the sorted part
  - Swap the item backwards until the item is in the right place within the sorted area (such that the left one  $<$  the item  $<$  the right one)

6 5 3 1 8 7 2 4

# Insertion sort - time complexity

while the size of the unsorted part is greater than 0:  
let the target element be the first element in the unsorted part

- find target's insertion point in the sorted part
- insert the target in its final, sorted position



- get a copy of the first element in the unsorted part
- while ( there are elements in the unsorted part to examine AND we haven't found the insertion point for the target ):
  - move the element up a position // make room for the target

|   | Worst    | Best        |
|---|----------|-------------|
| A | $O(n)$   | $O(n)$      |
| B | $O(n)$   | $O(\log n)$ |
| C | $O(n^2)$ | $O(n)$      |
| D | $O(n^2)$ | $O(n^2)$    |

**Class[Buzz]: selection\_sort**

# Insertion sort wins over selection sort

- In most cases the insertion sort is the best among the elementary sorts.
- Still executes in  $O(n^2)$  time as worst-case and somewhat faster than the selection sort in normal situations.
- Not too complex, although it's slightly more so than selection sort.
- Often used as the final stage of more sophisticated sorts, such as quicksort.

# Merge sort

- Split items into 2 roughly even pieces.
- Merge-sort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.

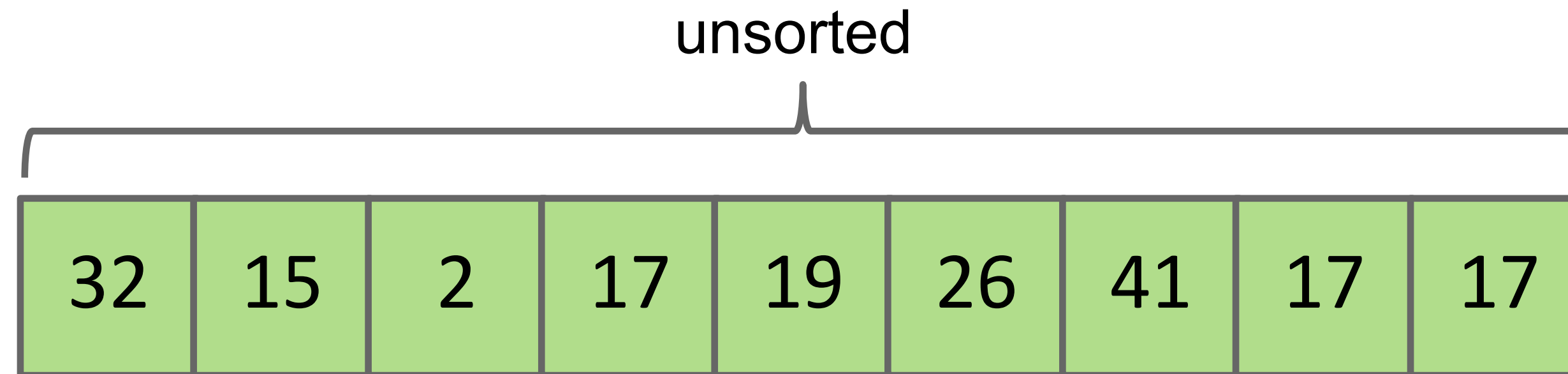
# Top-Down Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half.
- Merge the two sorted halves to form the final result.

Input:



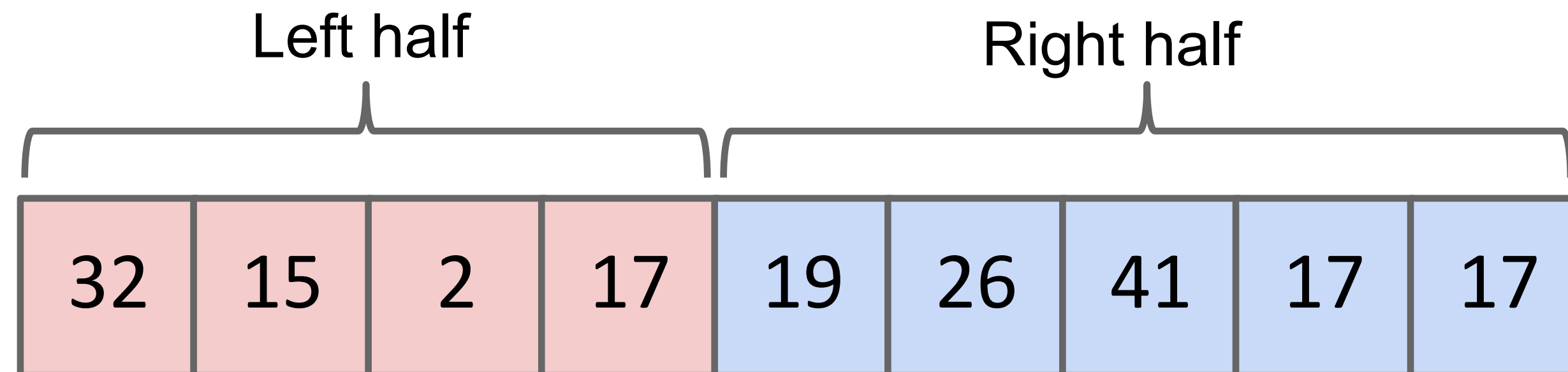
# Top-Down Merge Sort

---

Top-Down merge sorting N items:

- **Split items into 2 roughly even pieces.**
- Mergesort each half.
- Merge the two sorted halves to form the final result.

Input:



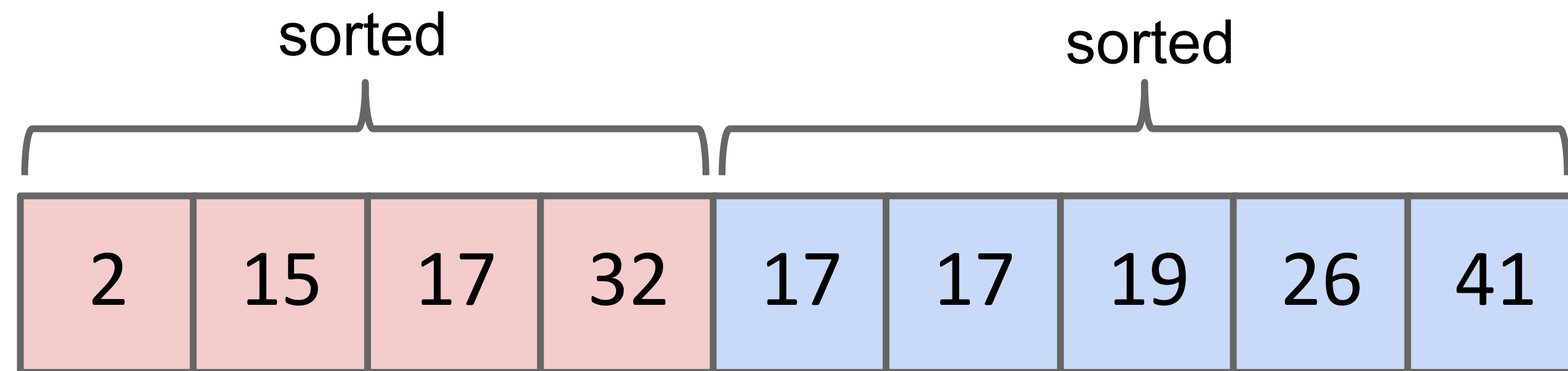
# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- **Mergesort each half (steps not shown, this is a recursive algorithm!)**
- Merge the two sorted halves to form the final result.

Input:



# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- **Merge the two sorted halves to form the final result.**
  - Compare  $\text{input}[i] < \text{input}[j]$ .
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
| i |    |    |    | j  |    |    |    |    |

Aux:

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| p |   |   |   |   |   |   |   |   |

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
| i |    |    |    | j  |    |    |    |    |

Aux:

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| p |   |   |   |   |   |   |   |   |

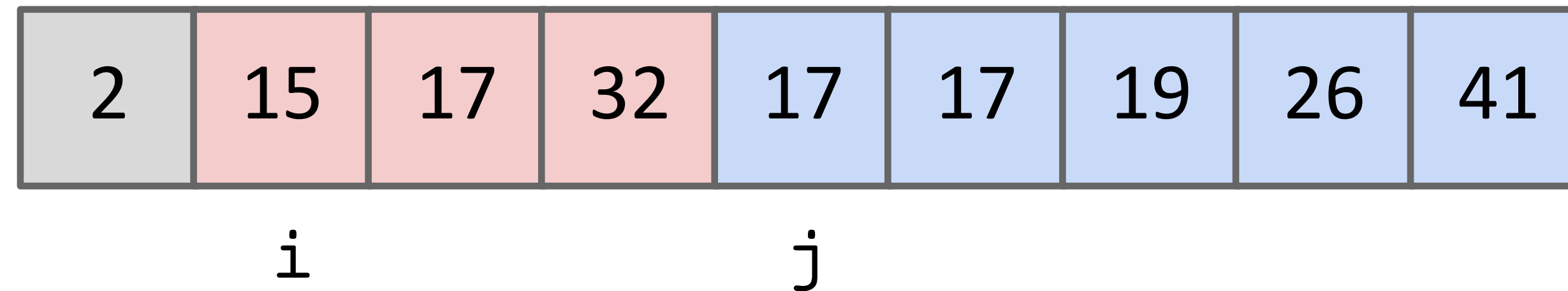
# Merge Sort

---

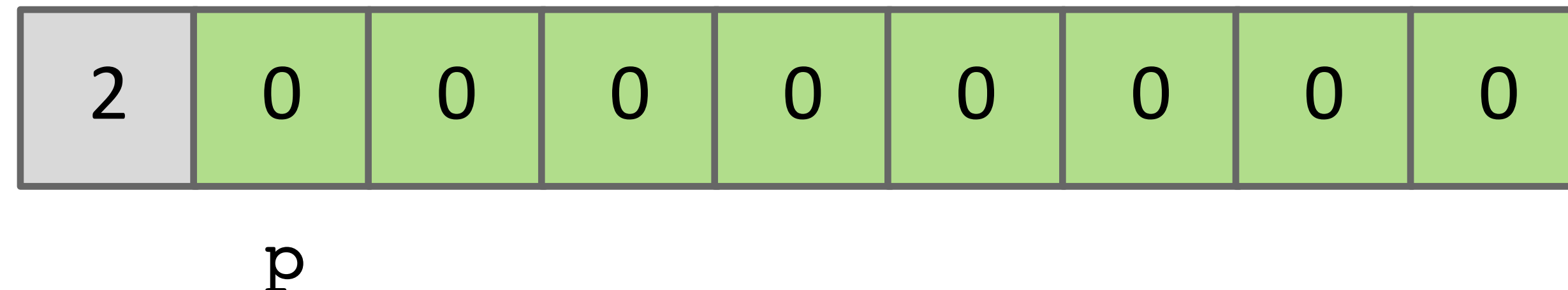
Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:



Aux:



# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   | i  |    |    | j  |    |    |    |    |

Aux:

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   | p |   |   |   |   |   |   |   |

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|---|----|----|----|----|----|----|----|----|

i

j

Aux:

|   |    |   |   |   |   |   |   |   |
|---|----|---|---|---|---|---|---|---|
| 2 | 15 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|---|----|---|---|---|---|---|---|---|

p

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    | i  |    | j  |    |    |    |    |

Aux:

|   |    |   |   |   |   |   |   |   |
|---|----|---|---|---|---|---|---|---|
| 2 | 15 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|   |    | p |   |   |   |   |   |   |

# Merge Sort

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|---|----|----|----|----|----|----|----|----|

i      j

Aux:

|   |    |    |   |   |   |   |   |   |
|---|----|----|---|---|---|---|---|---|
| 2 | 15 | 17 | 0 | 0 | 0 | 0 | 0 | 0 |
|---|----|----|---|---|---|---|---|---|

p

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    |    | i  | j  |    |    |    |    |

Aux:

|   |    |    |   |   |   |   |   |   |
|---|----|----|---|---|---|---|---|---|
| 2 | 15 | 17 | 0 | 0 | 0 | 0 | 0 | 0 |
|   |    |    | p |   |   |   |   |   |

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    |    | i  |    | j  |    |    |    |

Aux:

|   |    |    |    |   |   |   |   |   |
|---|----|----|----|---|---|---|---|---|
| 2 | 15 | 17 | 17 | 0 | 0 | 0 | 0 | 0 |
|   |    |    |    | p |   |   |   |   |

# Merge Sort

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    |    | i  |    | j  |    |    |    |

Aux:

|   |    |    |    |   |   |   |   |   |
|---|----|----|----|---|---|---|---|---|
| 2 | 15 | 17 | 17 | 0 | 0 | 0 | 0 | 0 |
|   |    |    |    | p |   |   |   |   |

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|---|----|----|----|----|----|----|----|----|

i

j

Aux:

|   |    |    |    |    |   |   |   |   |
|---|----|----|----|----|---|---|---|---|
| 2 | 15 | 17 | 17 | 17 | 0 | 0 | 0 | 0 |
|---|----|----|----|----|---|---|---|---|

p

# Merge Sort

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|---|----|----|----|----|----|----|----|----|

i

j

Aux:

|   |    |    |    |    |   |   |   |   |
|---|----|----|----|----|---|---|---|---|
| 2 | 15 | 17 | 17 | 17 | 0 | 0 | 0 | 0 |
|---|----|----|----|----|---|---|---|---|

p

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|---|----|----|----|----|----|----|----|----|

i

j

Aux:

|   |    |    |    |    |   |   |   |   |
|---|----|----|----|----|---|---|---|---|
| 2 | 15 | 17 | 17 | 17 | 0 | 0 | 0 | 0 |
|---|----|----|----|----|---|---|---|---|

p

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    |    | i  |    |    |    | j  |    |

Aux:

|   |    |    |    |    |    |   |   |   |
|---|----|----|----|----|----|---|---|---|
| 2 | 15 | 17 | 17 | 17 | 19 | 0 | 0 | 0 |
|   |    |    |    |    |    | p |   |   |

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    |    | i  |    |    |    | j  |    |

Aux:

|   |    |    |    |    |    |   |   |   |
|---|----|----|----|----|----|---|---|---|
| 2 | 15 | 17 | 17 | 17 | 19 | 0 | 0 | 0 |
|   |    |    |    |    |    | p |   |   |

# Merge Sort

# Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

# Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    |    | i  |    |    |    |    | j  |

Aux:

|   |    |    |    |    |    |    |   |   |
|---|----|----|----|----|----|----|---|---|
| 2 | 15 | 17 | 17 | 17 | 19 | 26 | 0 | 0 |
|---|----|----|----|----|----|----|---|---|

p

# Merge Sort

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|   |    |    | i  |    |    |    |    | j  |

Aux:

|   |    |    |    |    |    |    |   |   |
|---|----|----|----|----|----|----|---|---|
| 2 | 15 | 17 | 17 | 17 | 19 | 26 | 0 | 0 |
|   |    |    |    |    |    |    | p |   |

# Merge Sort

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - **Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).**
  - Copy smaller item and increment p and i or j.

No comparison is made this time, since the left side has run out of items!

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|---|----|----|----|----|----|----|----|----|

i

j

Aux:

|   |    |    |    |    |    |    |    |   |
|---|----|----|----|----|----|----|----|---|
| 2 | 15 | 17 | 17 | 17 | 19 | 26 | 32 | 0 |
|---|----|----|----|----|----|----|----|---|

p

# Merge Sort

---

Top-Down merge sorting N items:

- Split items into 2 roughly even pieces.
- Mergesort each half (steps not shown, this is a recursive algorithm!)
- Merge the two sorted halves to form the final result.
  - Compare  $\text{input}[i] < \text{input}[j]$  (if necessary).
  - **Copy smaller item and increment p and i or j.**

Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 32 | 17 | 17 | 19 | 26 | 41 |
|---|----|----|----|----|----|----|----|----|

i

j

Aux:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 17 | 17 | 19 | 26 | 32 | 41 |
|---|----|----|----|----|----|----|----|----|

p

# Merge Sort

Time complexity of merge sort?

|   | Worst         | Best          |
|---|---------------|---------------|
| A | $O(n)$        | $O(1)$        |
| B | $O(n \log n)$ | $O(n \log n)$ |
| C | $O(n)$        | $O(\log n)$   |
| D | $O(n^2)$      | $O(n)$        |

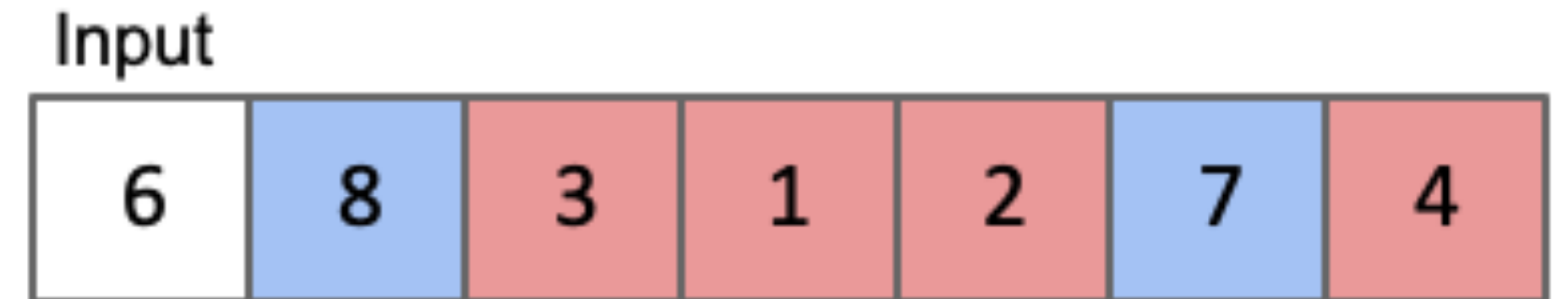
**Class[Buzz]: merge\_sort**

# Quick sort uses “partitioning”

- To partition a list ‘a’ on element at index i,
  - Define the partitioning pivot x ( $a[i]$  in this case)
  - Rearrange all elements so that
    - the left of x are  $\leq x$ .
    - the right of x are  $\geq x$ .

# Partitioning takes $O(n)$

- Algorithm:
  - Create another list.
  - Scan and copy one item at a time from the left
    - if red, copy it to the leftmost empty spot
    - if blue, copy it to the rightmost empty spot
    - copy the white item to the last empty spot



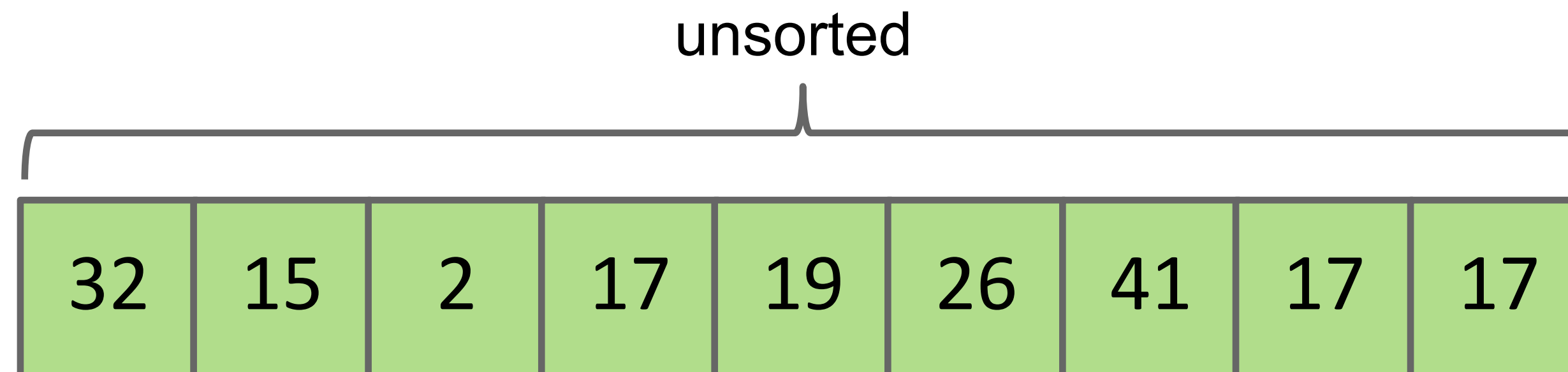
# Quick Sort

---

Quick sorting N items:

- Partition on leftmost item.
- Quicksort left half.
- Quicksort right half.

Input:



# Quick Sort

---

Quick sorting N items:

**partition(32)**

- **Partition on leftmost item (32).**
- Quicksort left half.
- Quicksort right half.

Input:

|    |    |   |    |    |    |    |    |    |
|----|----|---|----|----|----|----|----|----|
| 32 | 15 | 2 | 17 | 19 | 26 | 41 | 17 | 17 |
|----|----|---|----|----|----|----|----|----|

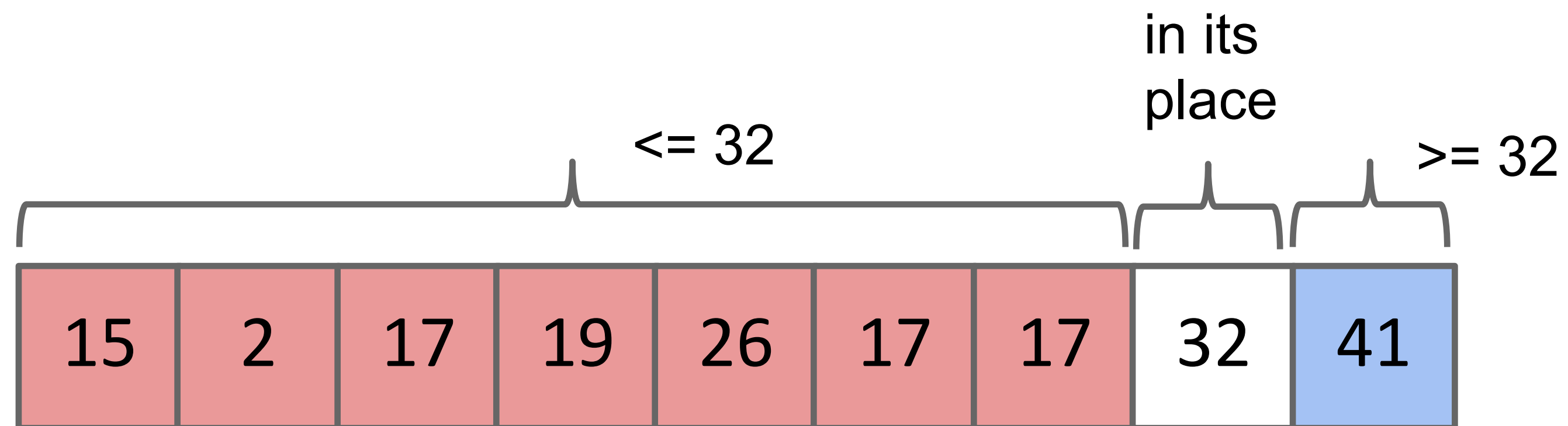
# Quick Sort

Quick sorting N items:

- **Partition on leftmost item (32) (done).**
- Quicksort left half.
- Quicksort right half.

**partition(32)**

Input:



# Quick Sort

---

Quick sorting N items:

- Partition on leftmost item (32) (done).
- **Quicksort left half (up next).**
- Quicksort right half.

partition(32)

Input:



# Quick Sort

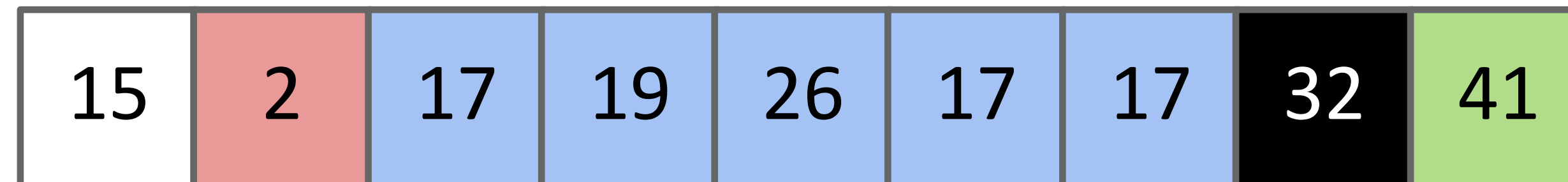
---

Quick sorting N items:

- **Partition on leftmost item (15).**
- Quicksort left half.
- Quicksort right half.

partition(32)  
partition(15)

Input:

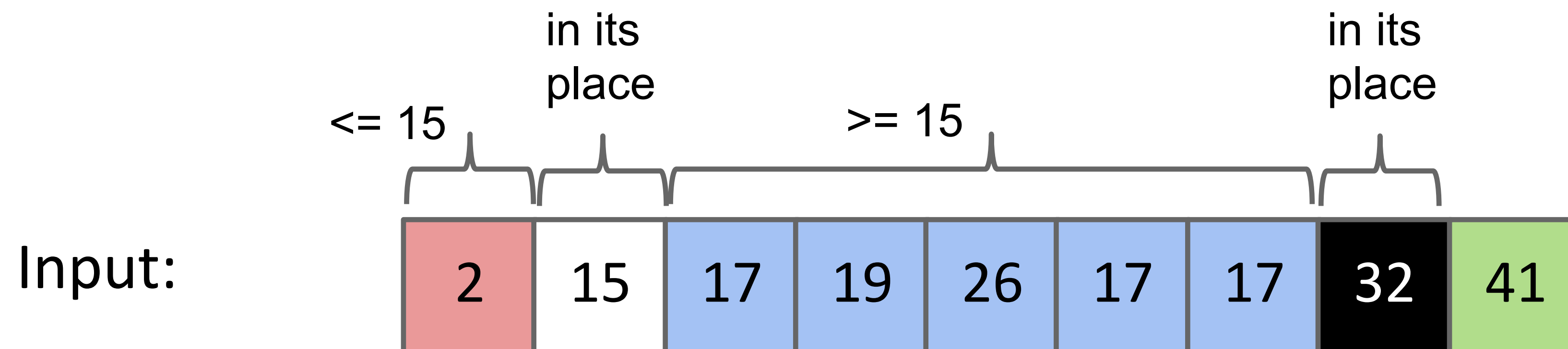


# Quick Sort

Quick sorting N items:

- **Partition on leftmost item (15) (done).**
- Quicksort left half.
- Quicksort right half.

partition(32)  
partition(15)



# Quick Sort

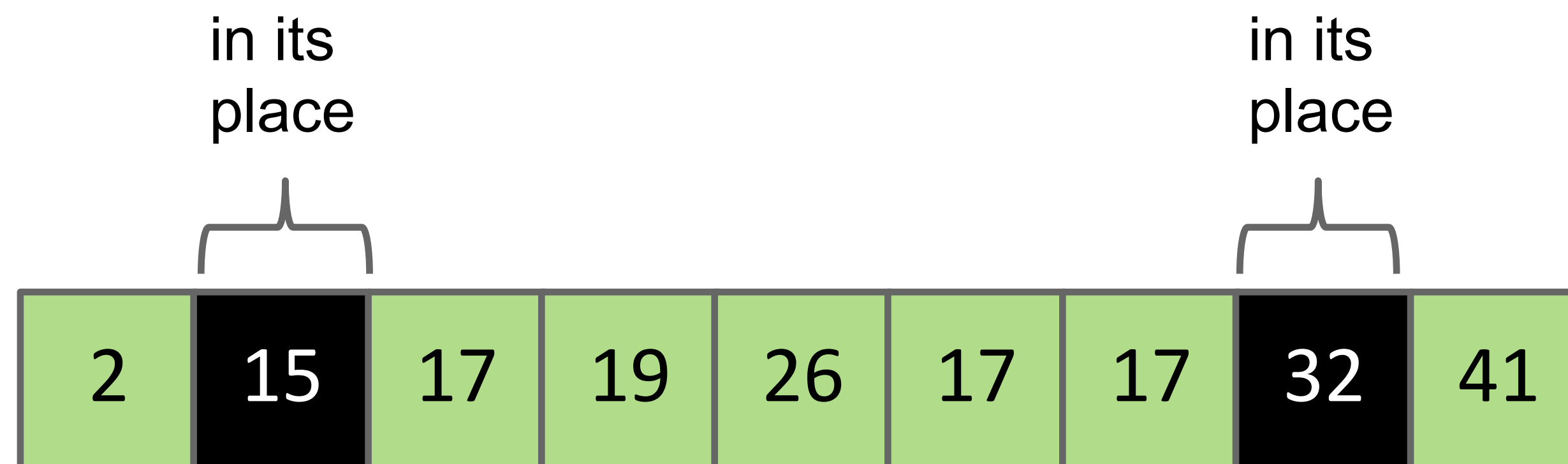
---

Quick sorting N items:

- Partition on leftmost item (15) (done).
- **Quicksort left half (up next).**
- Quicksort right half.

partition(32)  
partition(15)

Input:



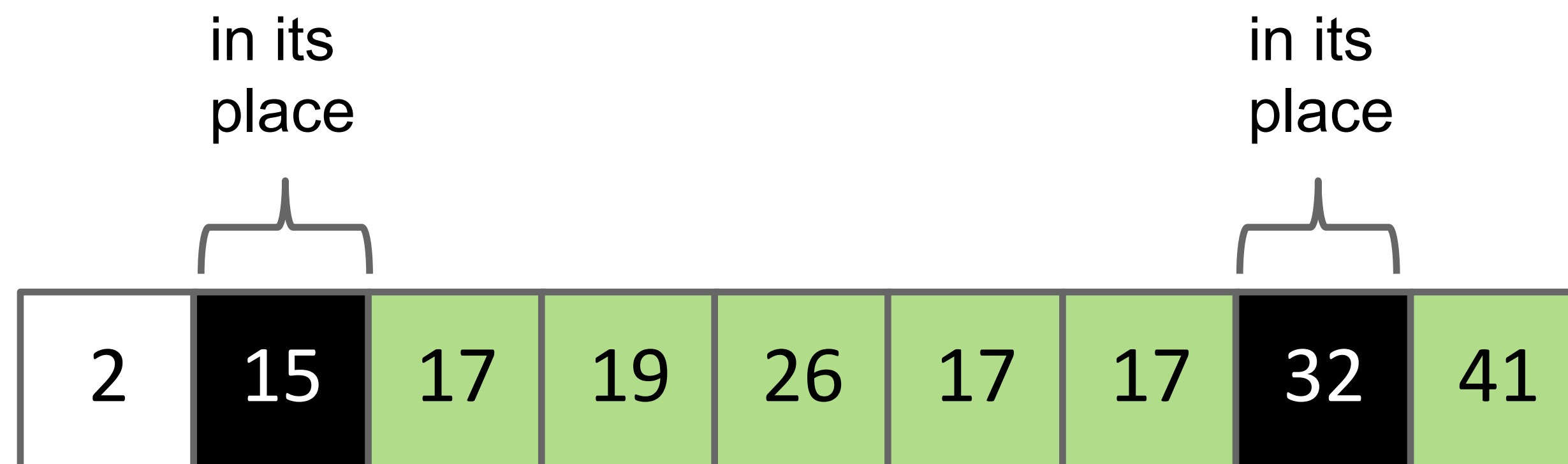
# Quick Sort

Quick sorting N items:

- **Partition on leftmost item (2).**
- Quicksort left half.
- Quicksort right half.

partition(32)  
partition(15)  
partition(2)

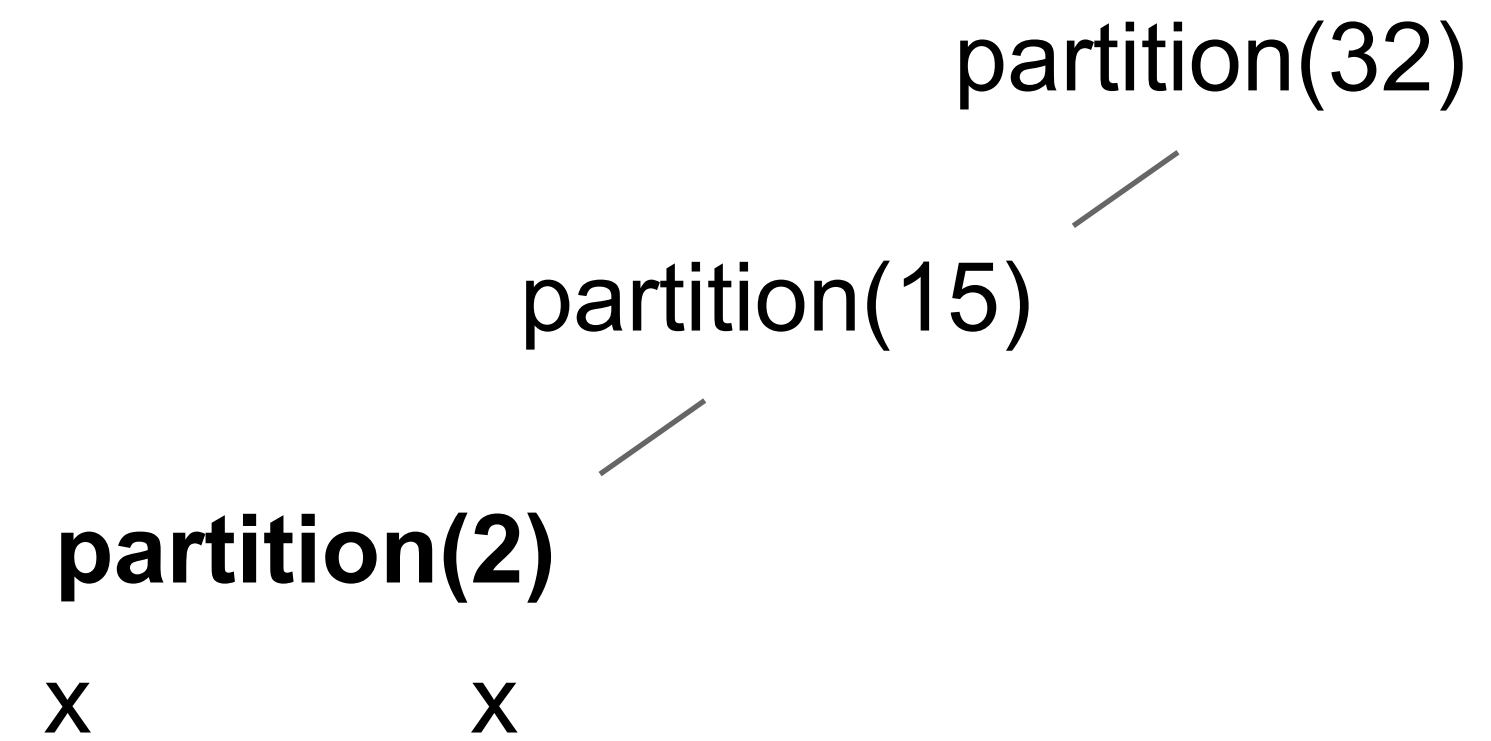
Input:



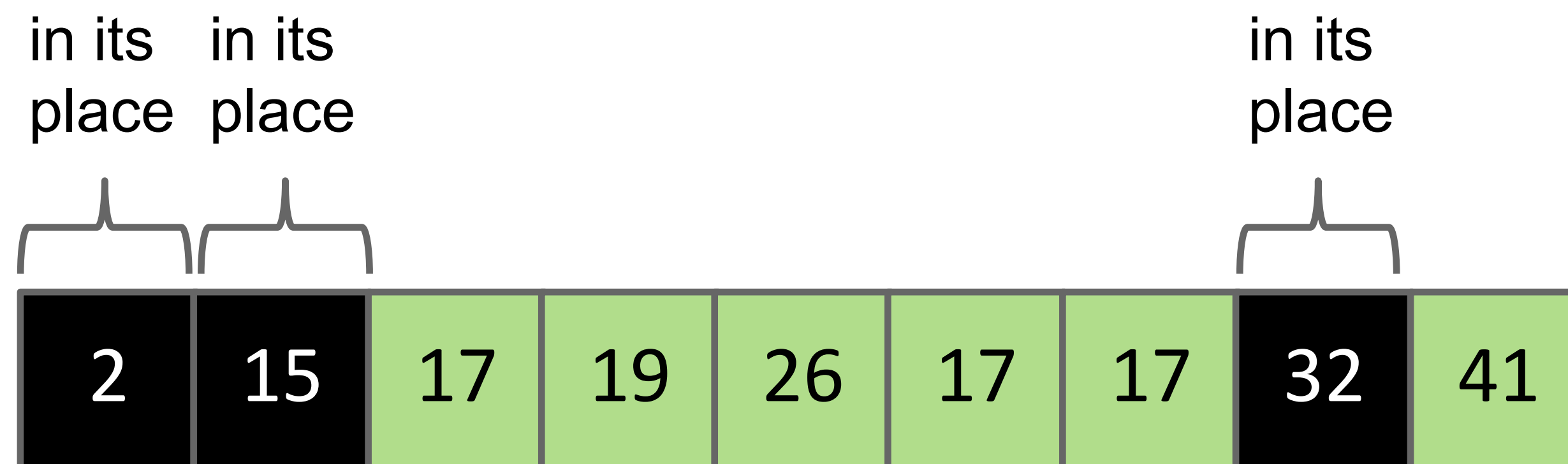
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (2) (done).
- **Quicksort left half (size zero, done).**
- **Quicksort right half (size zero, done).**



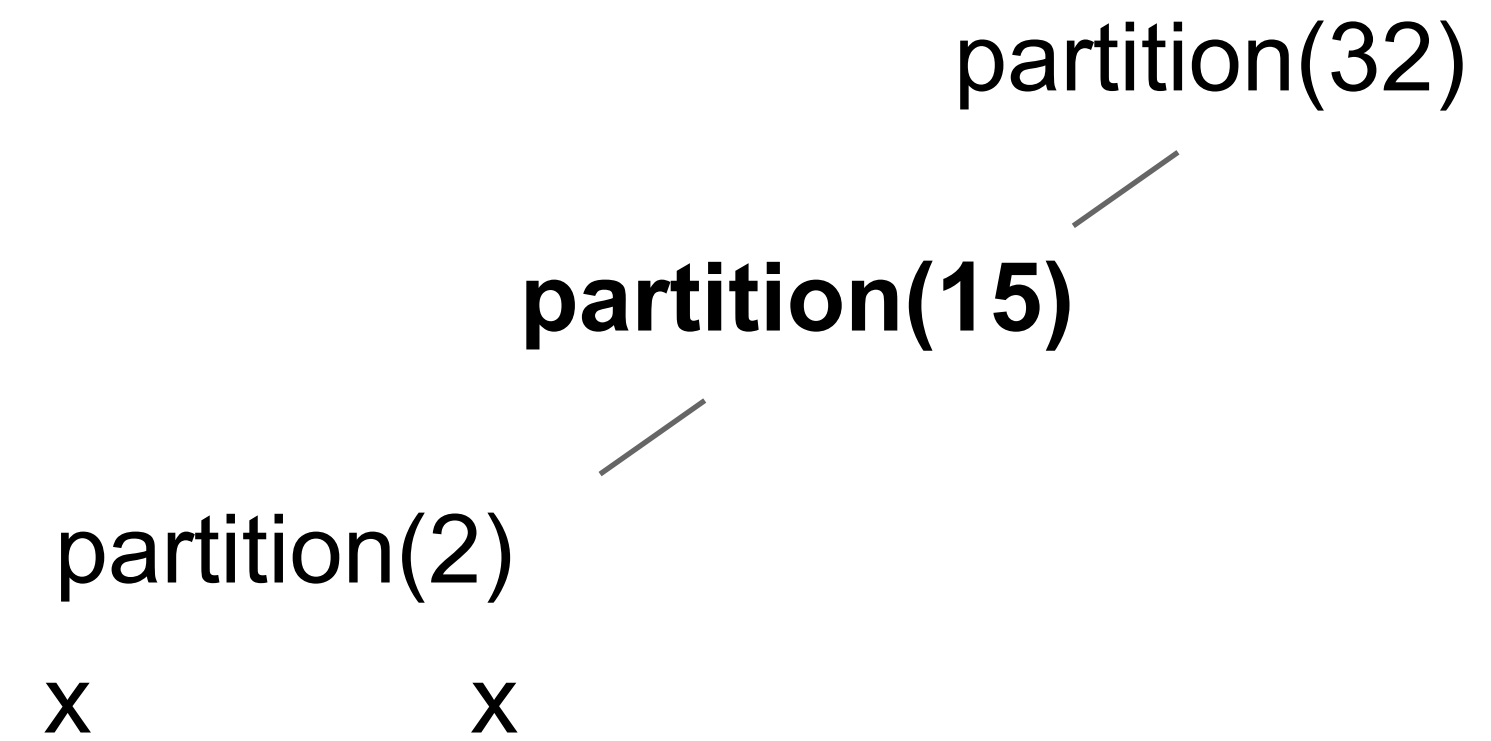
Input:



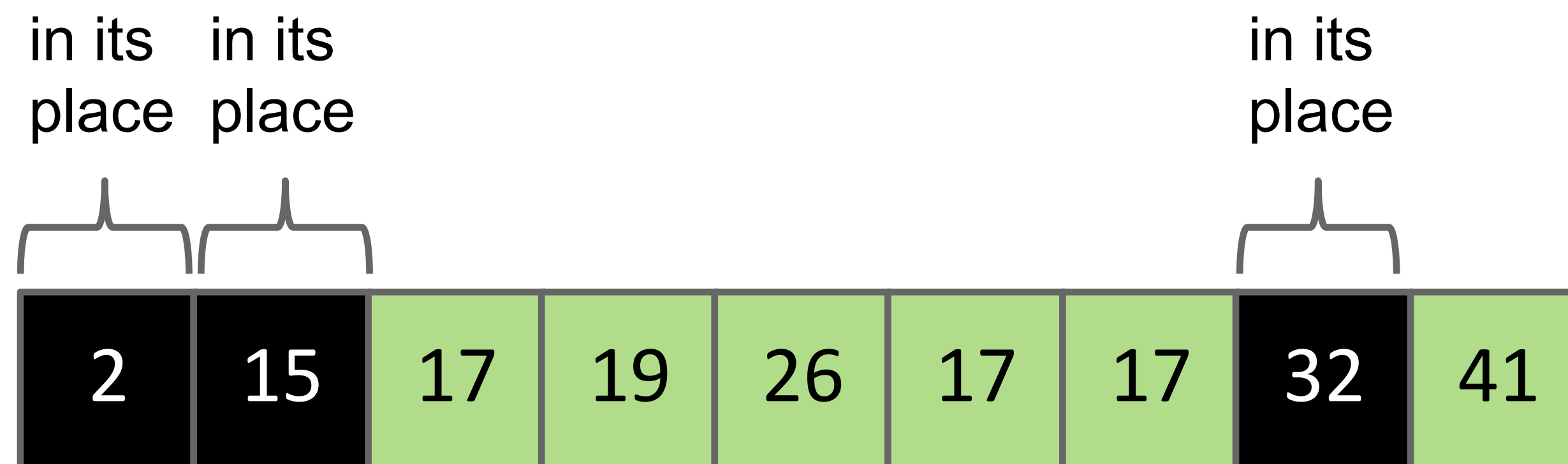
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (2) (done).
- Quicksort left half (done).
- **Quicksort right half (up next).**



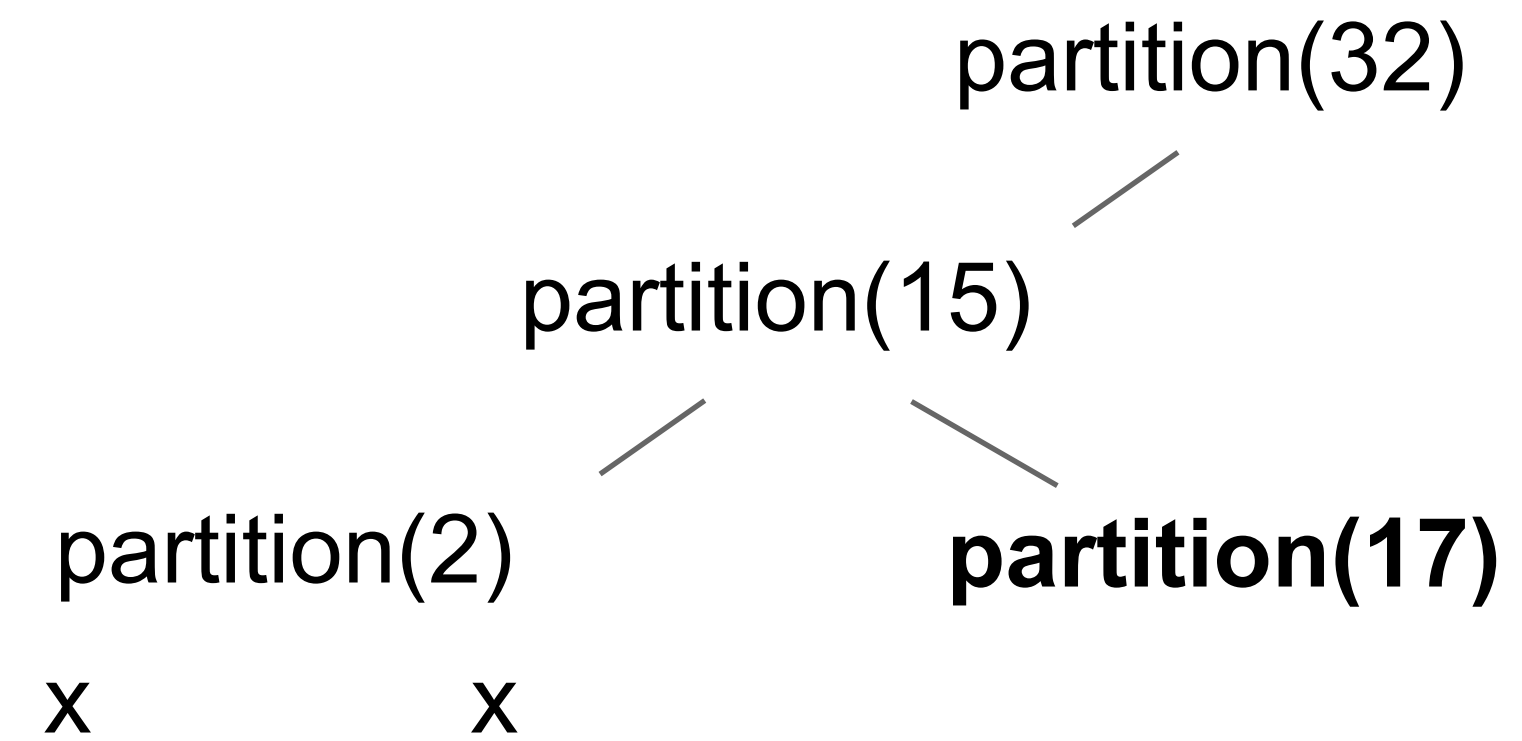
Input:



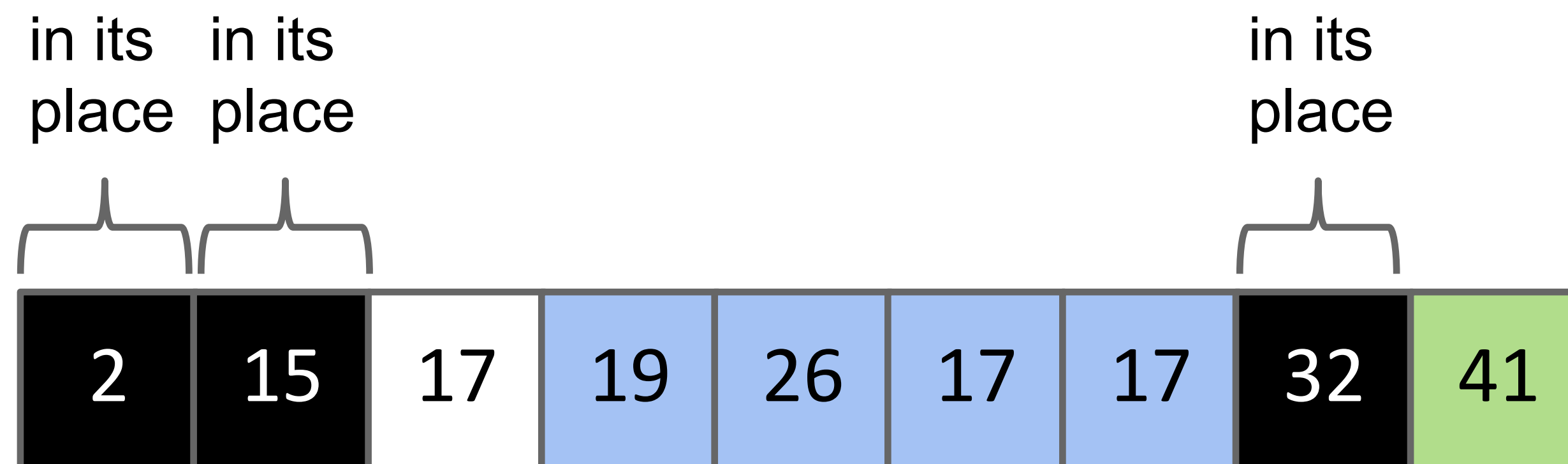
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (17)
- Quicksort left half.
- Quicksort right half.



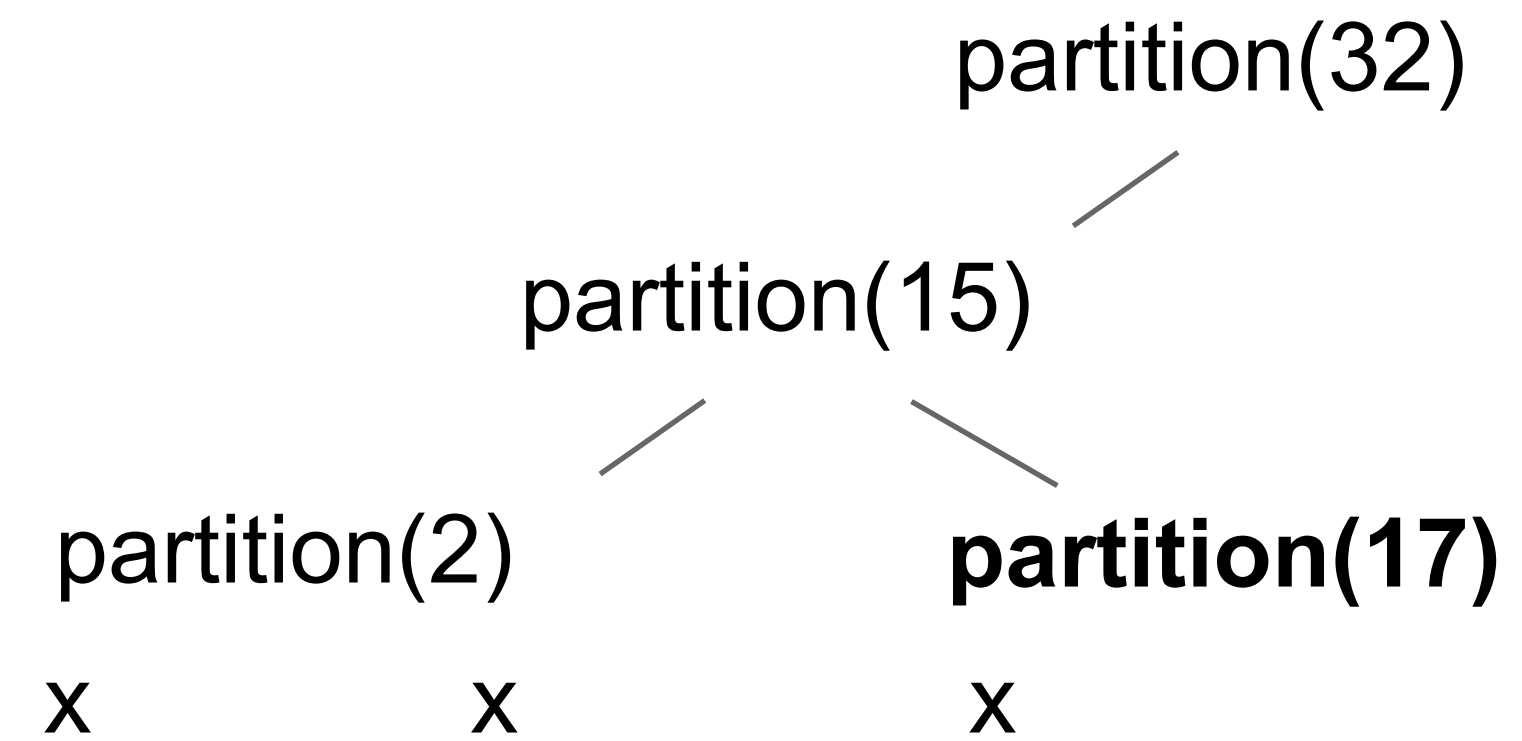
Input:



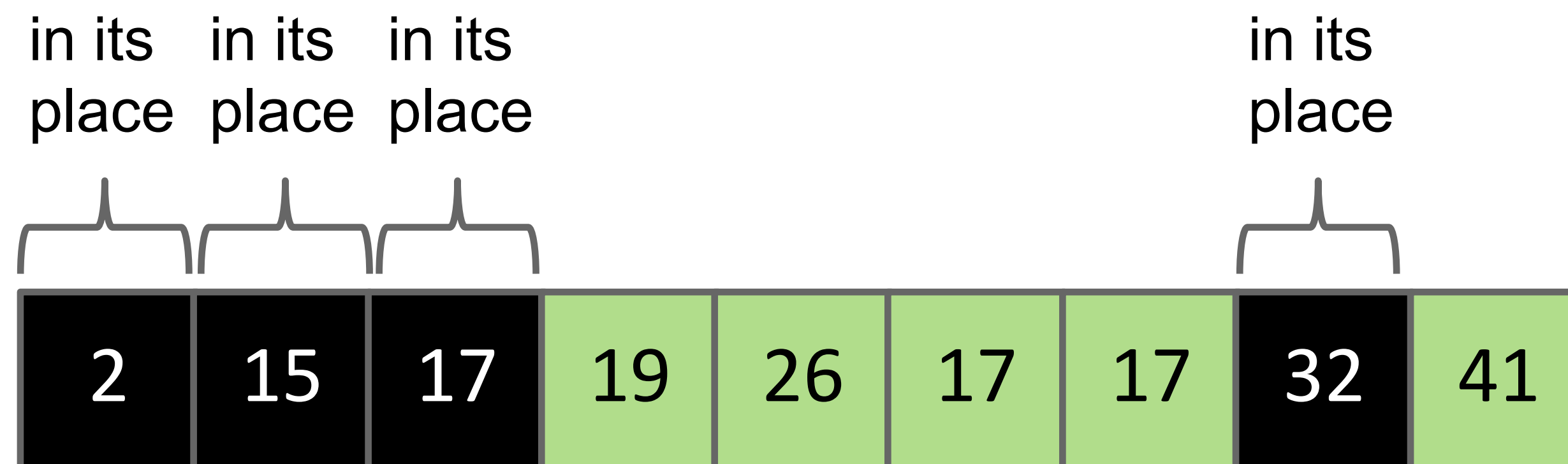
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (17) (done).
- **Quicksort left half (size zero, done).**
- **Quicksort right half (up next).**



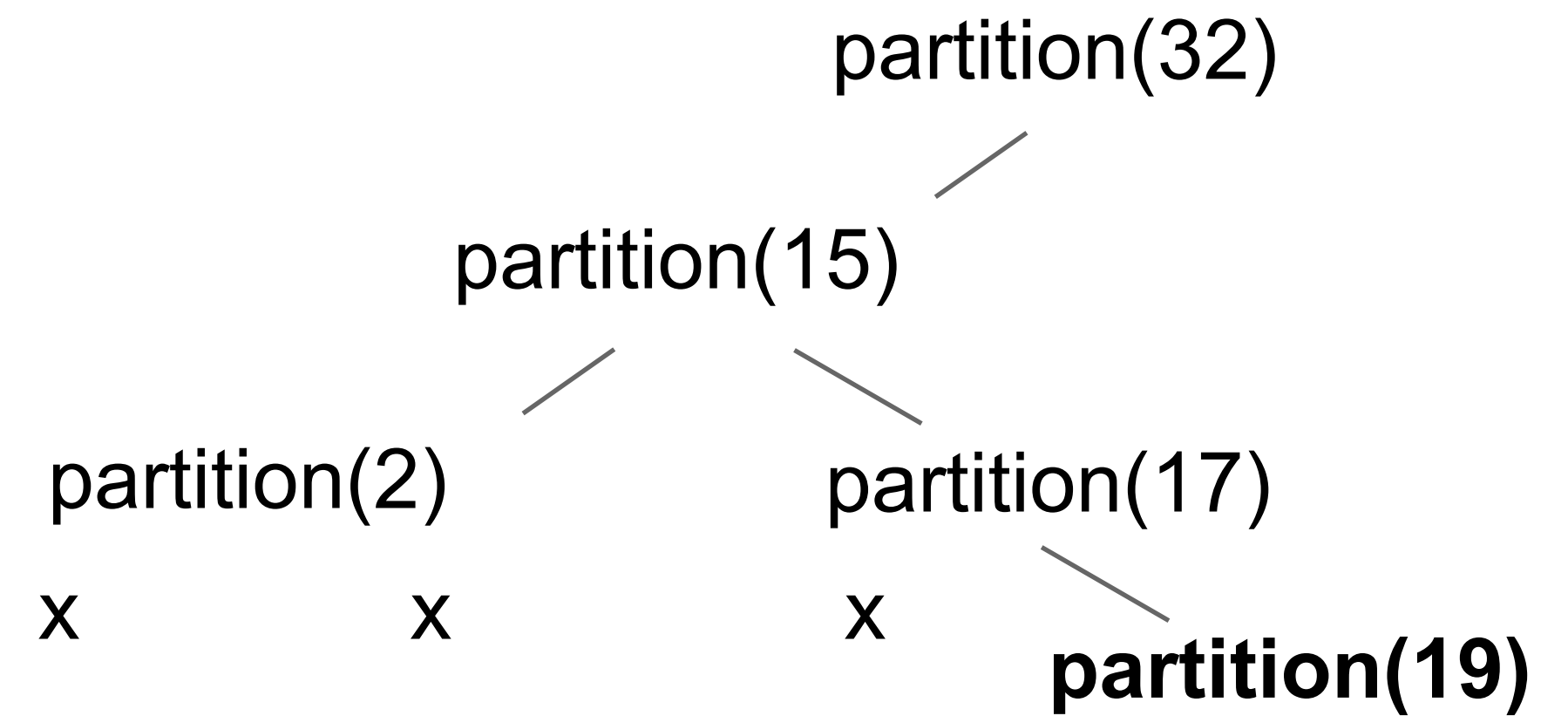
Input:



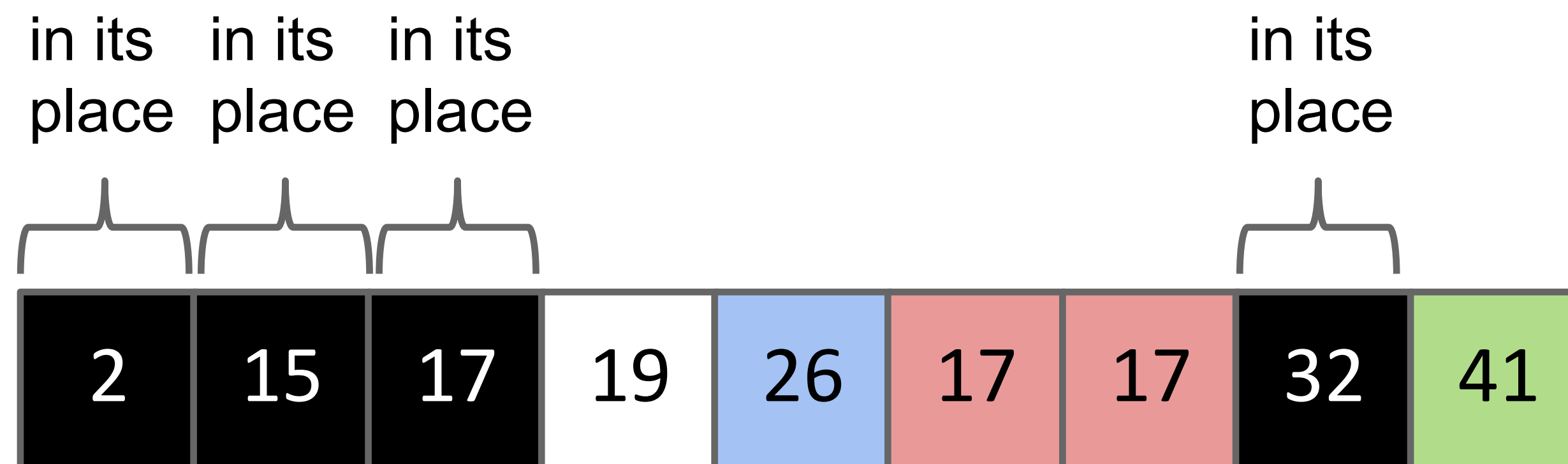
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (19).
- Quicksort left half.
- Quicksort right half.



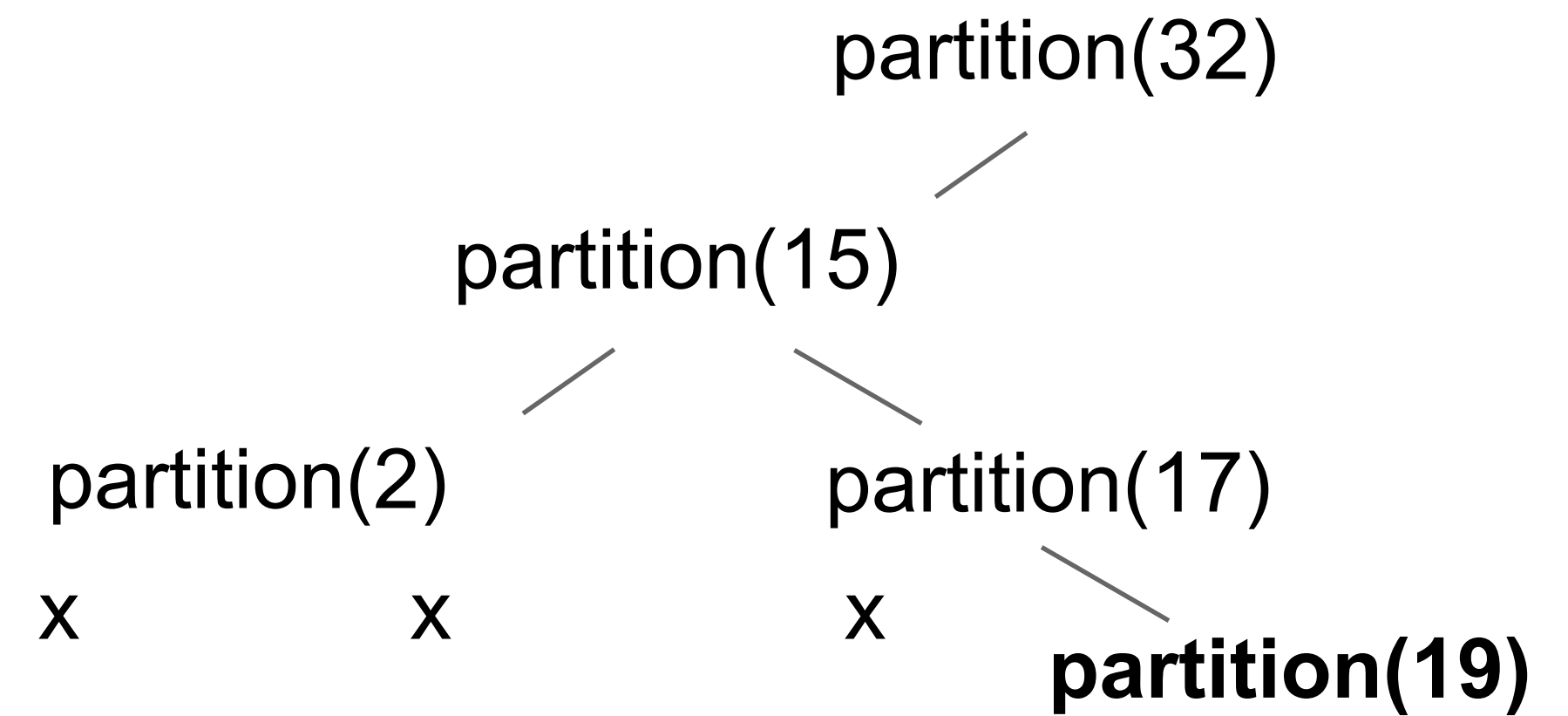
Input:



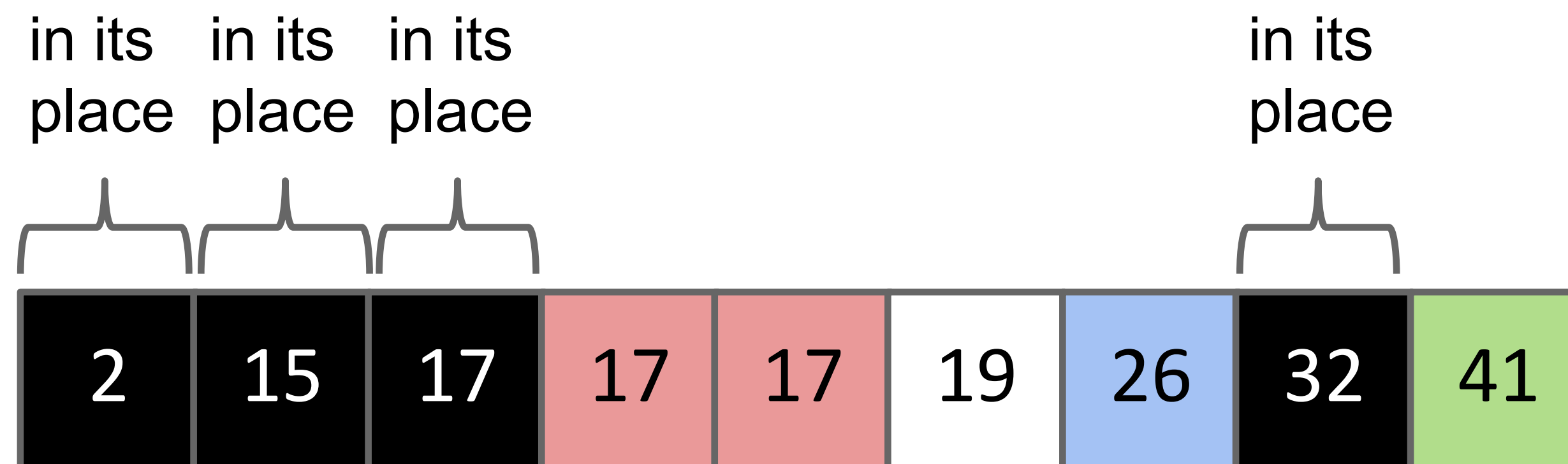
# Quick Sort

Quick sorting N items:

- **Partition on leftmost item (19) (done).**
- Quicksort left half.
- Quicksort right half.



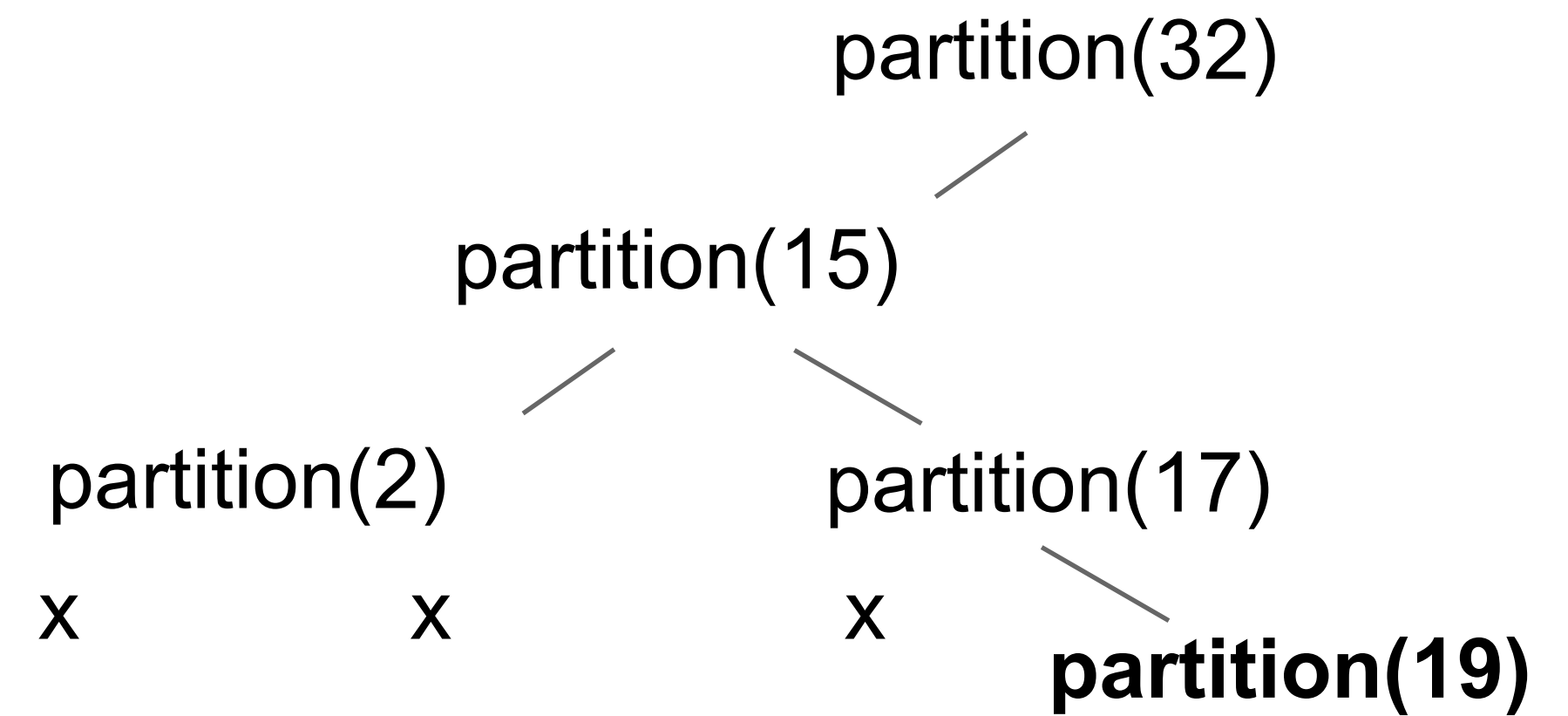
Input:



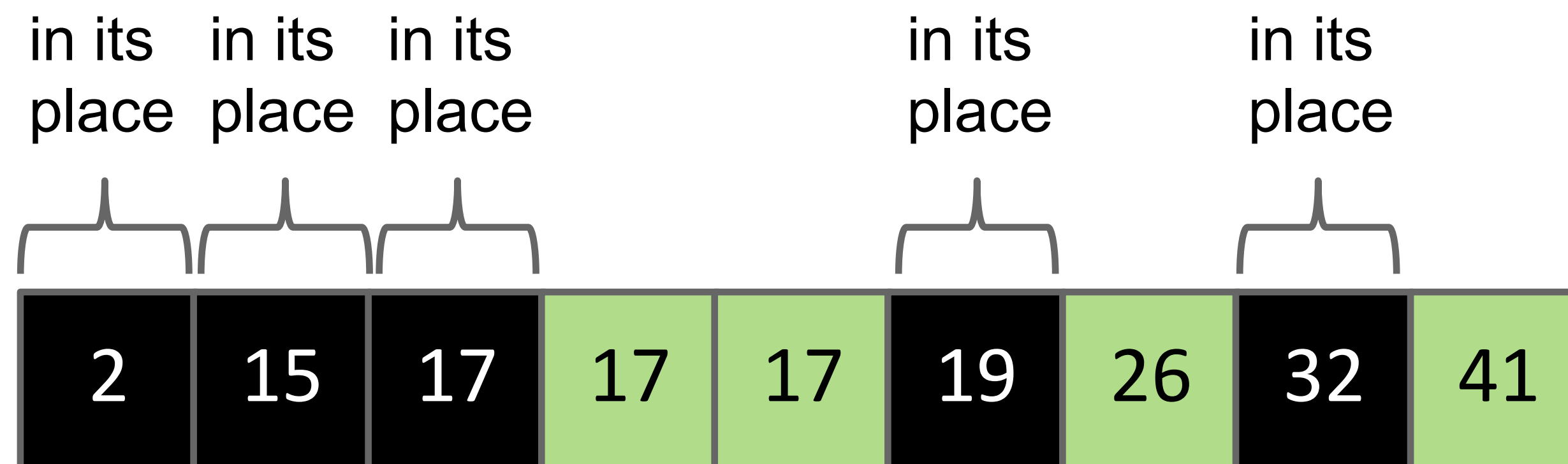
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (19).
- **Quicksort left half (up next).**
- Quicksort right half.



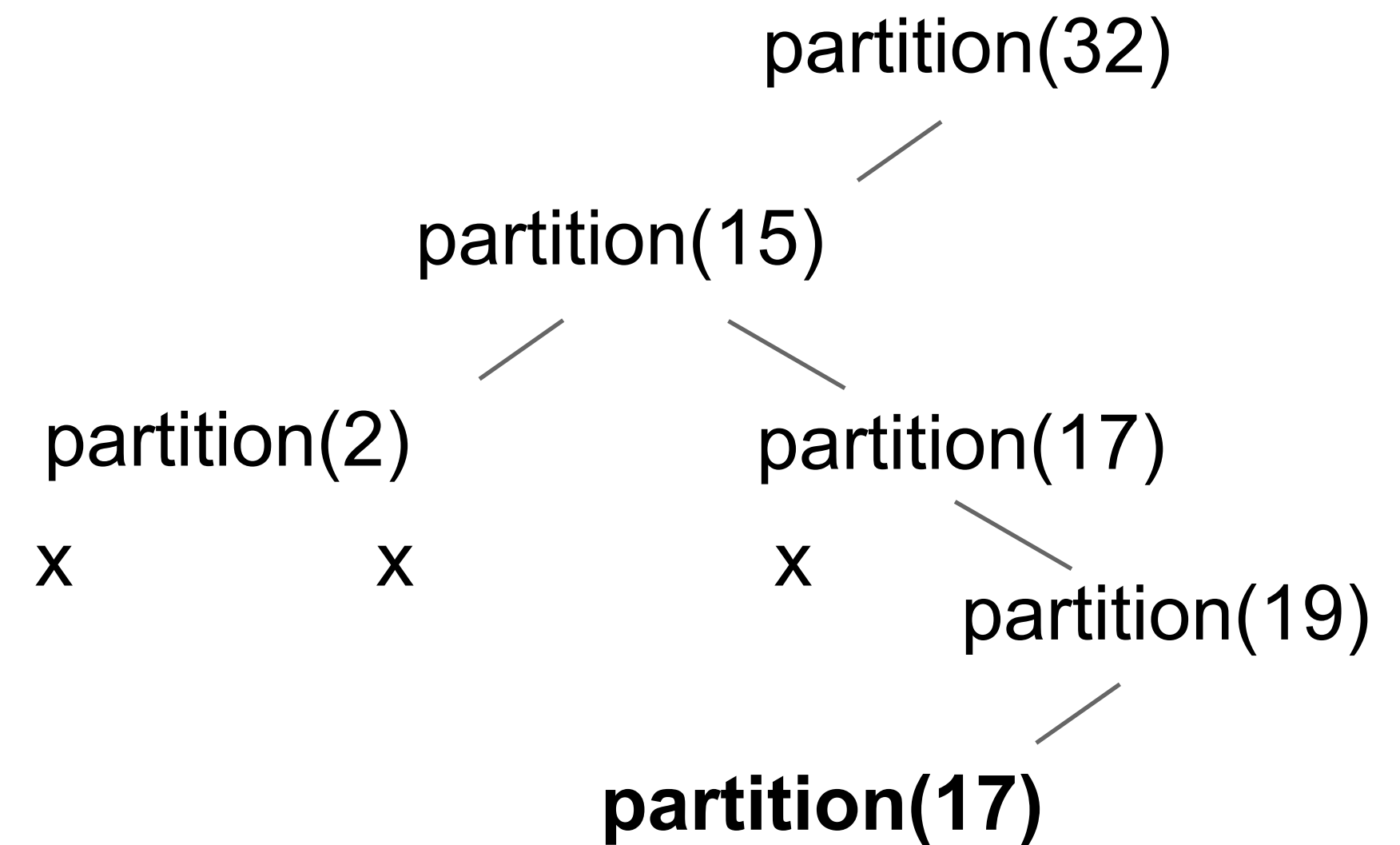
Input:



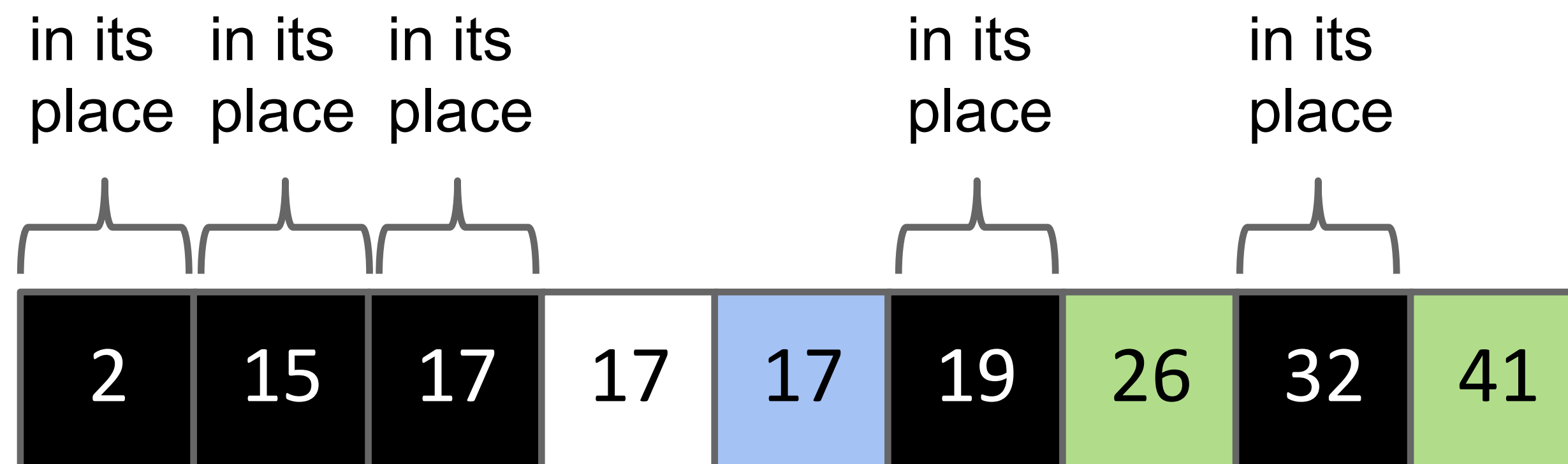
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (17).
- Quicksort left half.
- Quicksort right half.



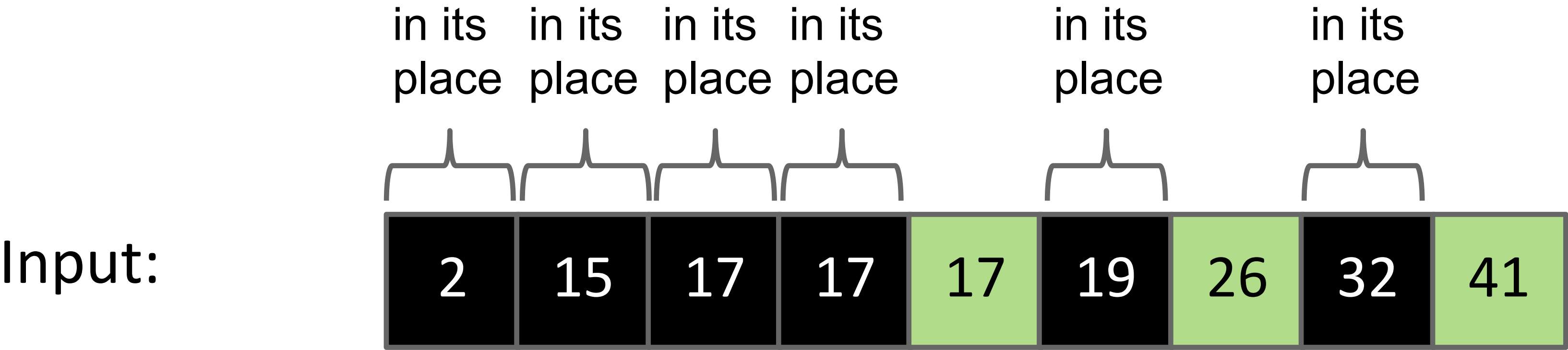
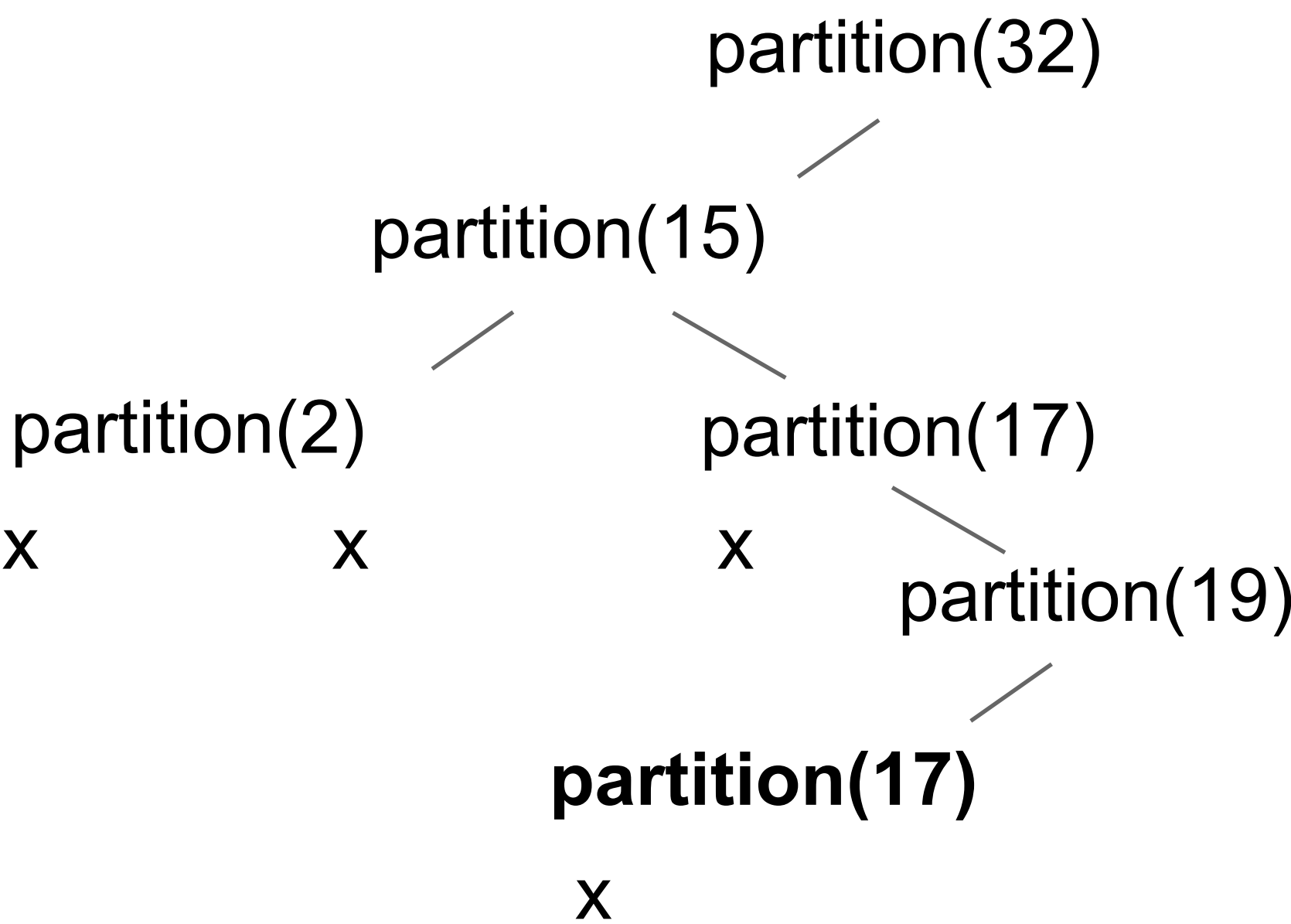
Input:



# Quick Sort

Quick sorting N items:

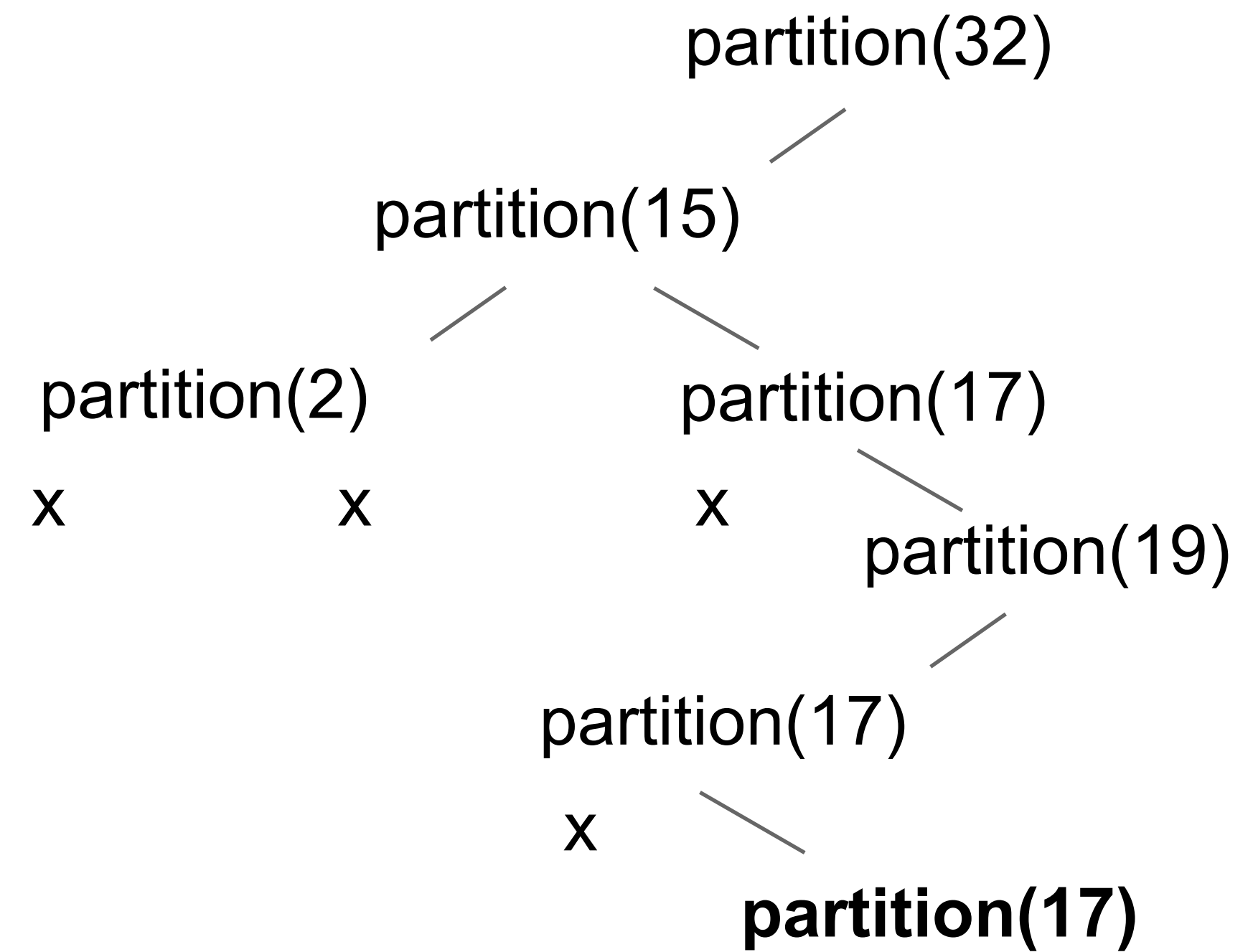
- Partition on leftmost item (17) (done).
- **Quicksort left half (size zero, done).**
- **Quicksort right half (up next).**



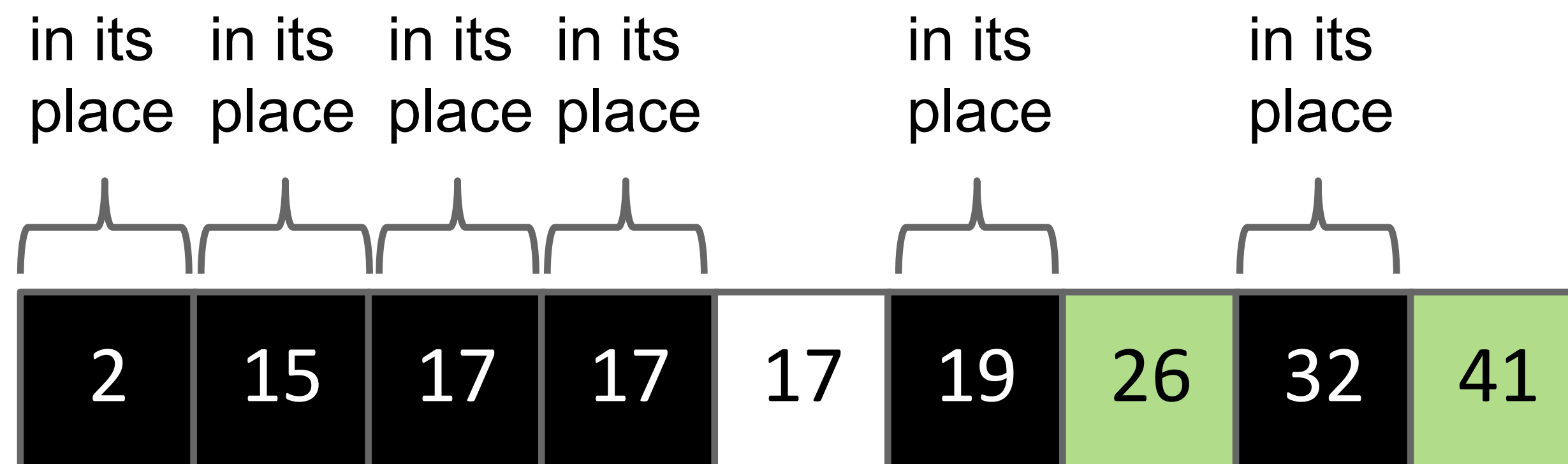
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (17).
- Quicksort left half.
- Quicksort right half.



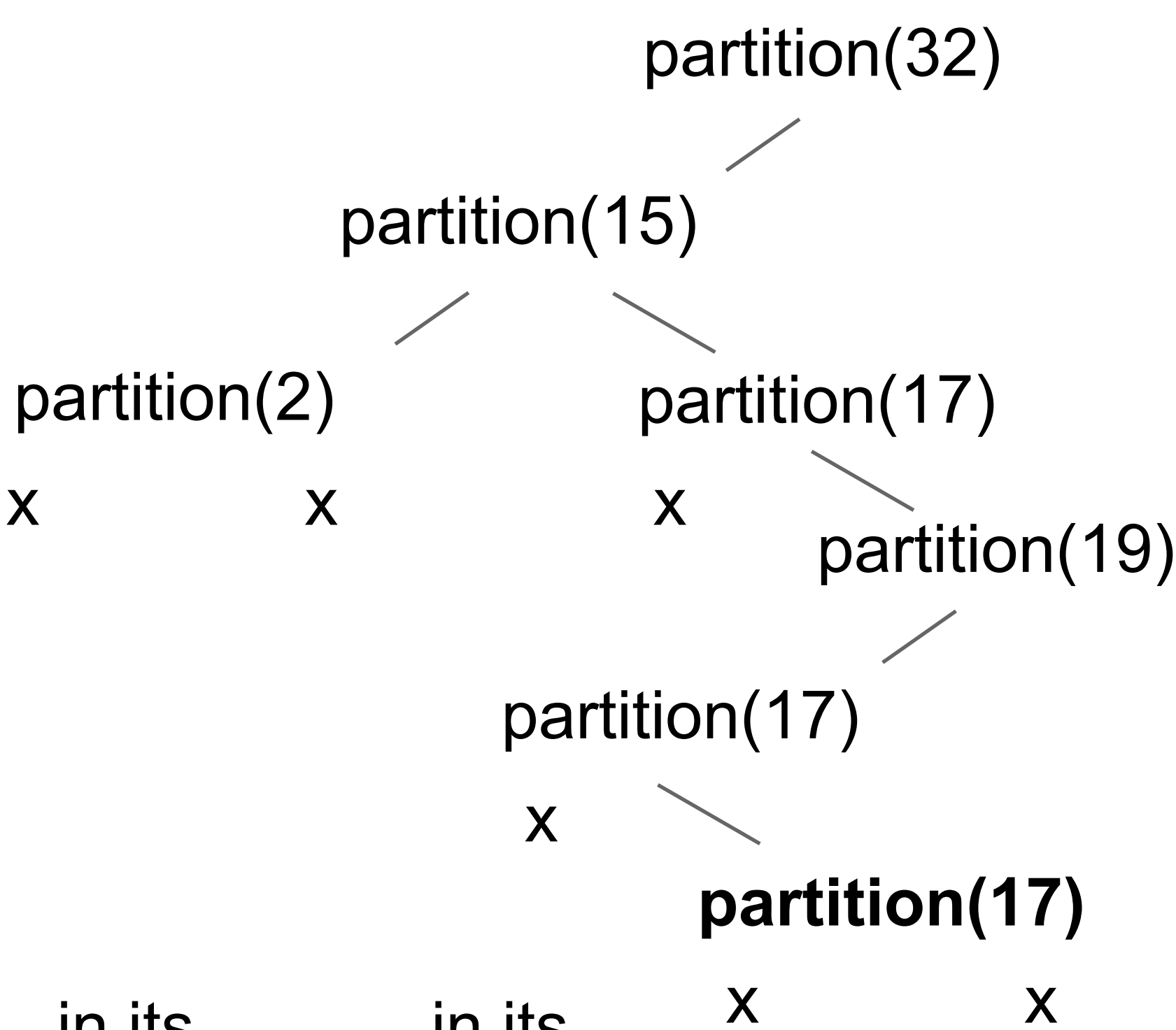
Input:



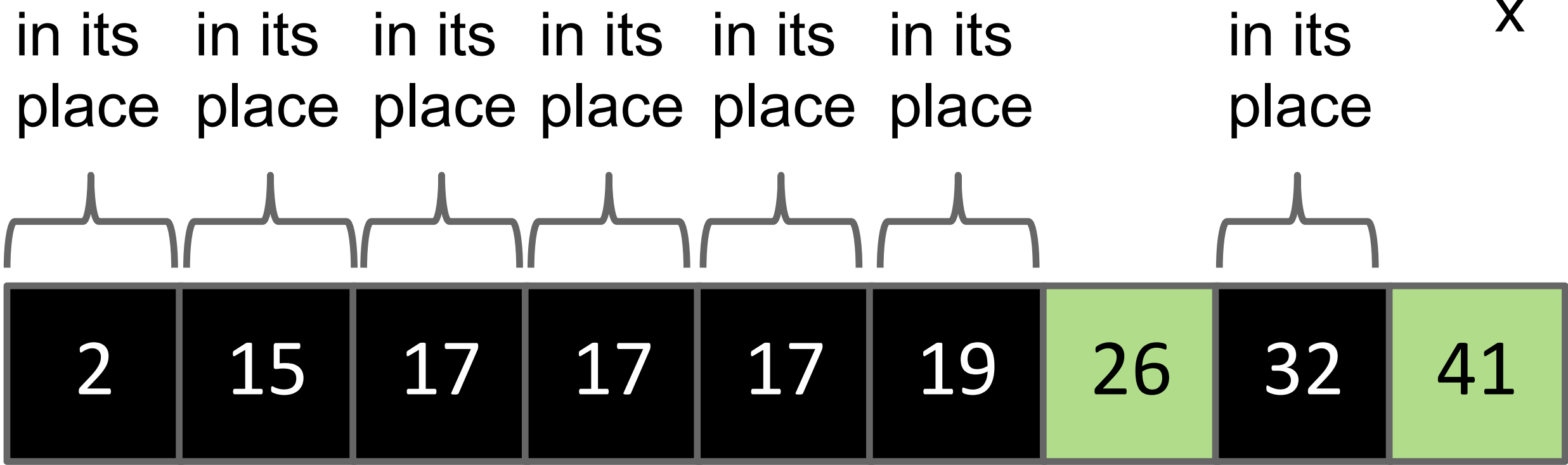
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (17) (done).
- **Quicksort left half (size zero, done).**
- **Quicksort right half (size zero, done).**



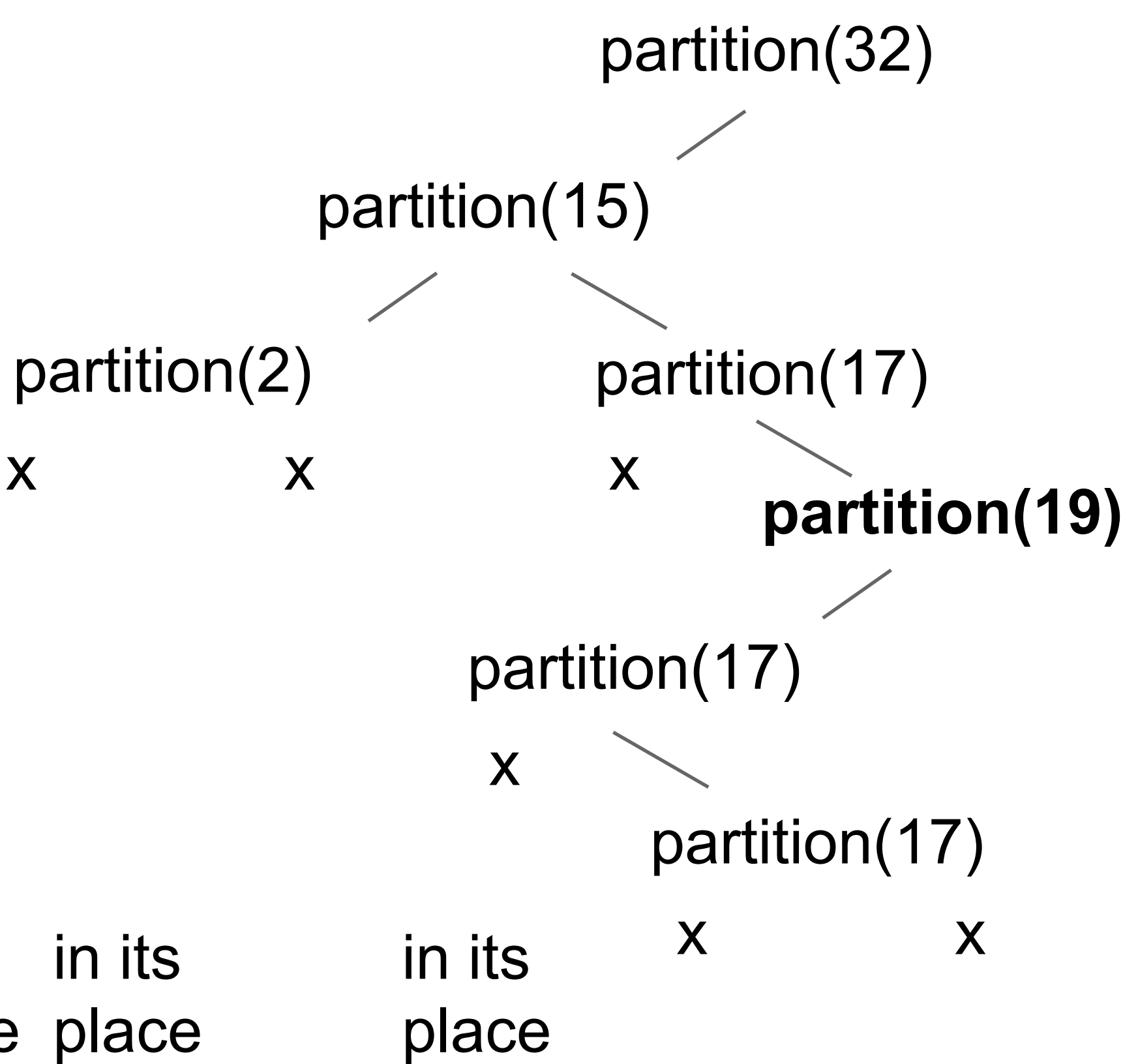
Input:



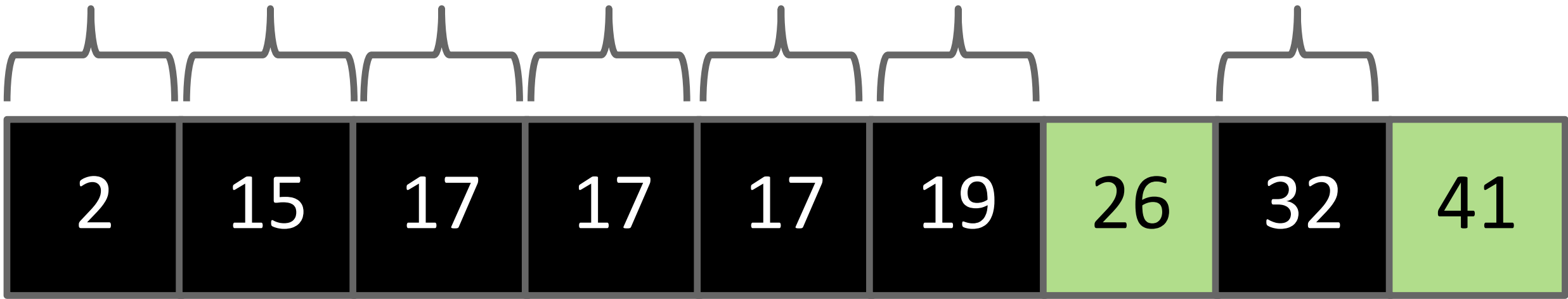
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (19) (done).
- Quicksort left half (done).
- **Quicksort right half (up next).**



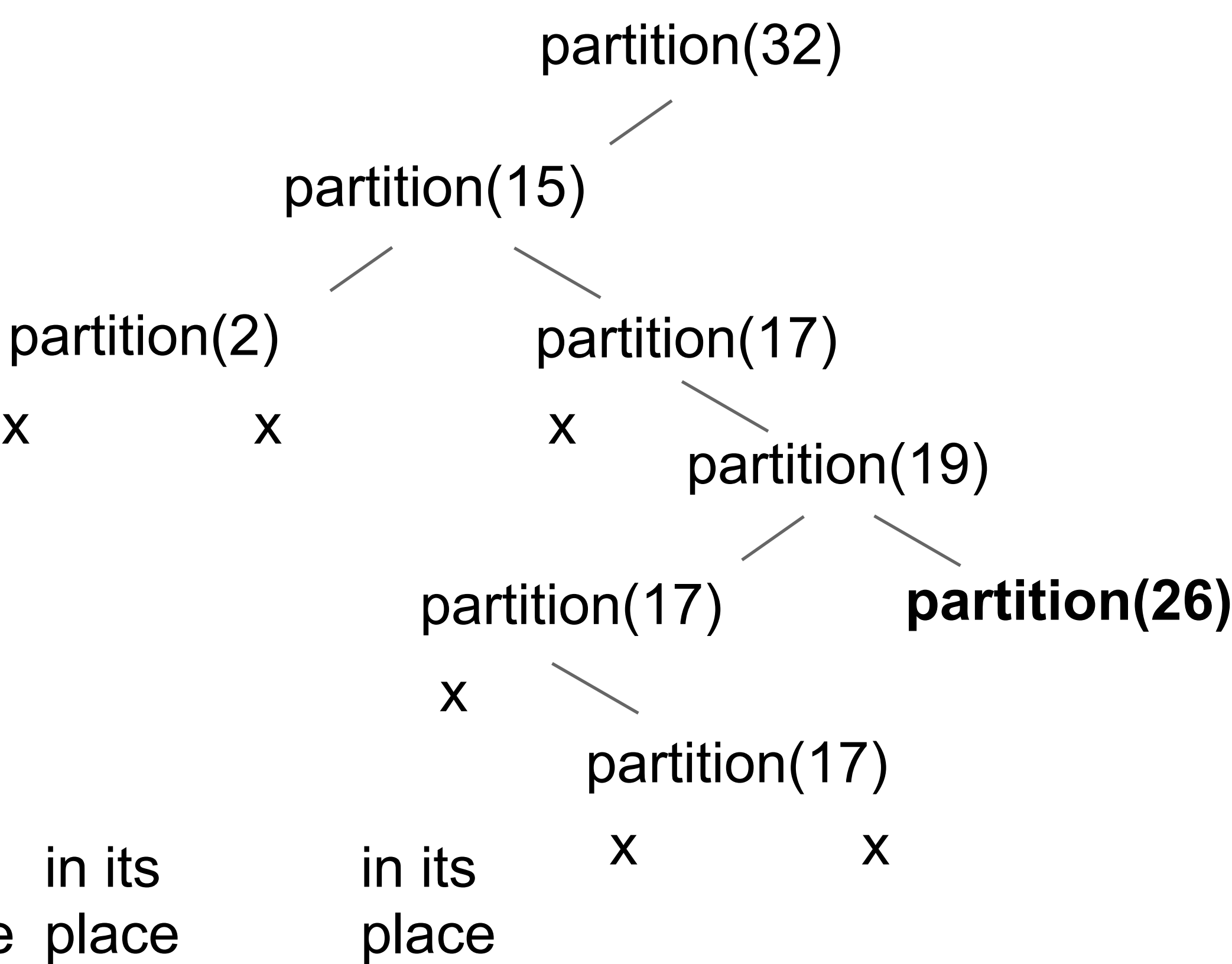
Input:



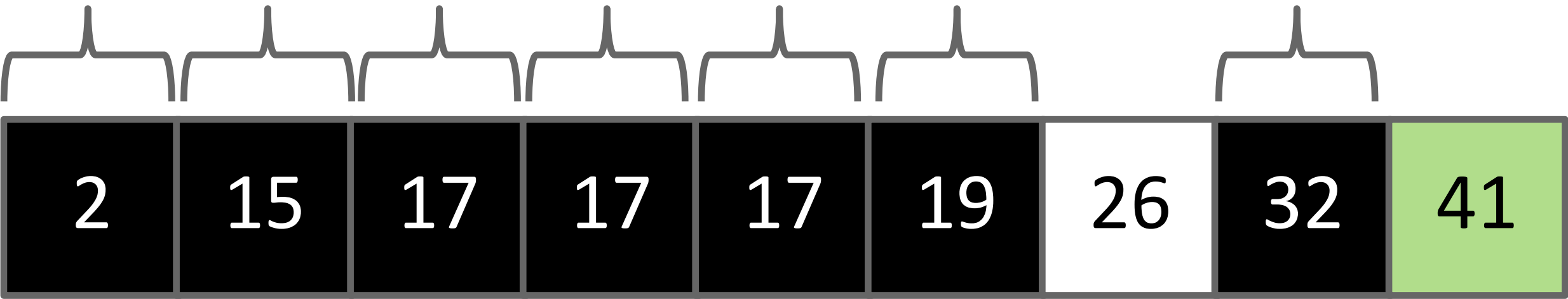
# Quick Sort

Quick sorting N items:

- Partition on leftmost item (26).
- Quicksort left half.
- Quicksort right half.



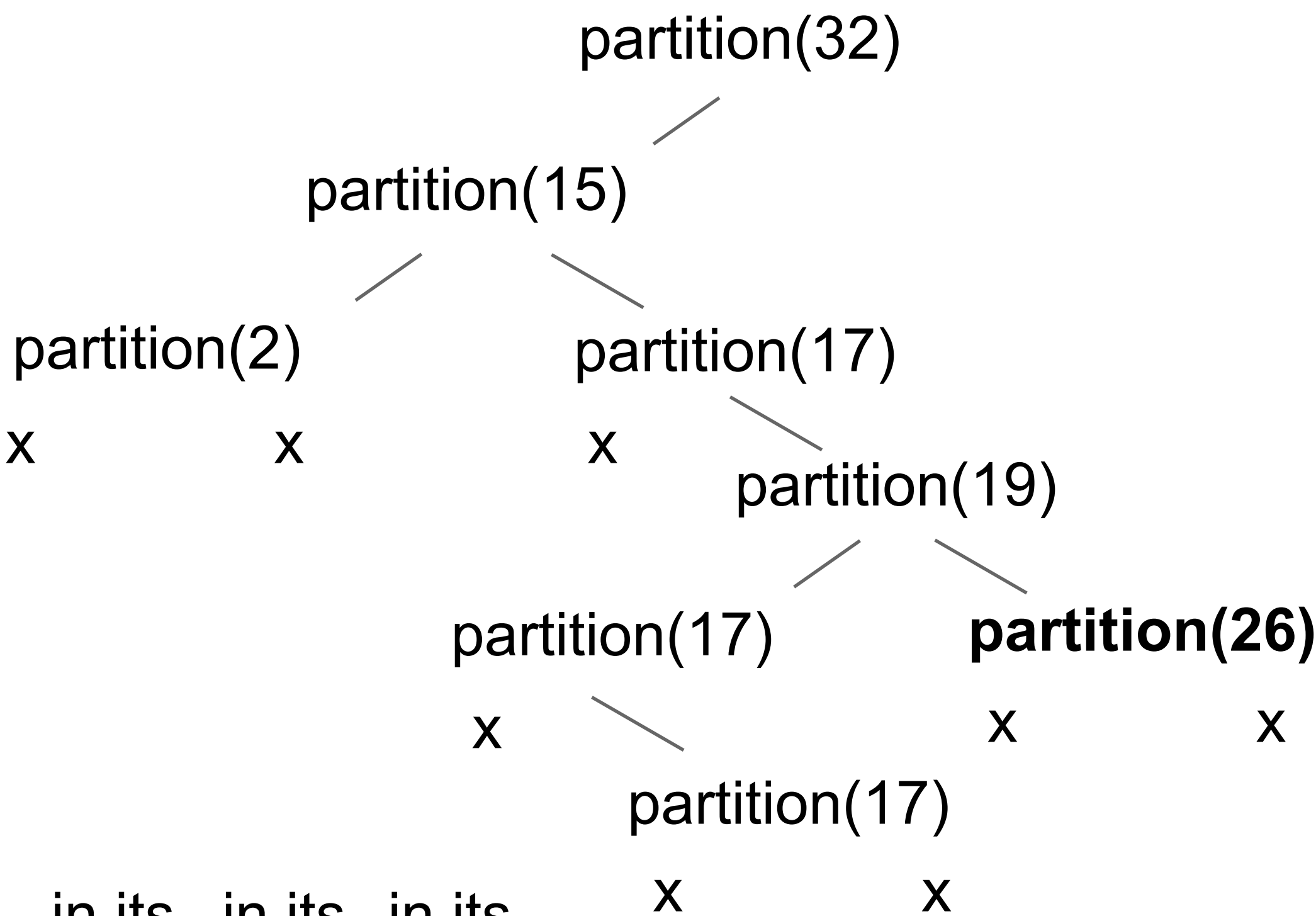
Input:



# Quick Sort

Quick sorting N items:

- Partition on leftmost item (26) (done).
- **Quicksort left half (size zero, done).**
- **Quicksort right half (size zero, done).**



in its place in its place in its place in its place in its place in its place in its place

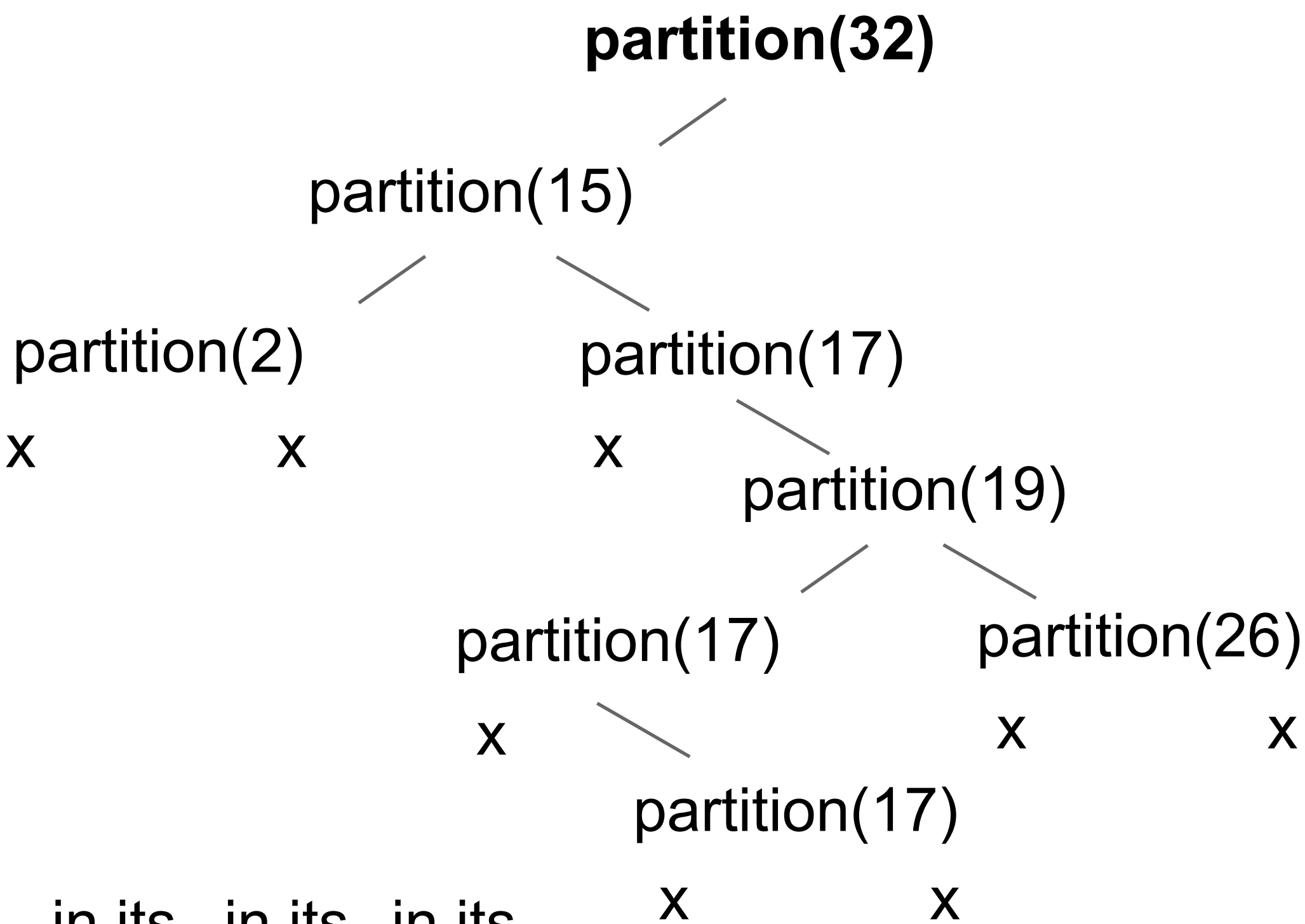
Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 17 | 17 | 19 | 26 | 32 | 41 |
|---|----|----|----|----|----|----|----|----|

# Quick Sort

Quick sorting N items:

- Partition on leftmost item (32) (done).
- Quicksort left half (done).
- **Quicksort right half (up next).**



in its place in its place in its place in its place in its place in its place in its place

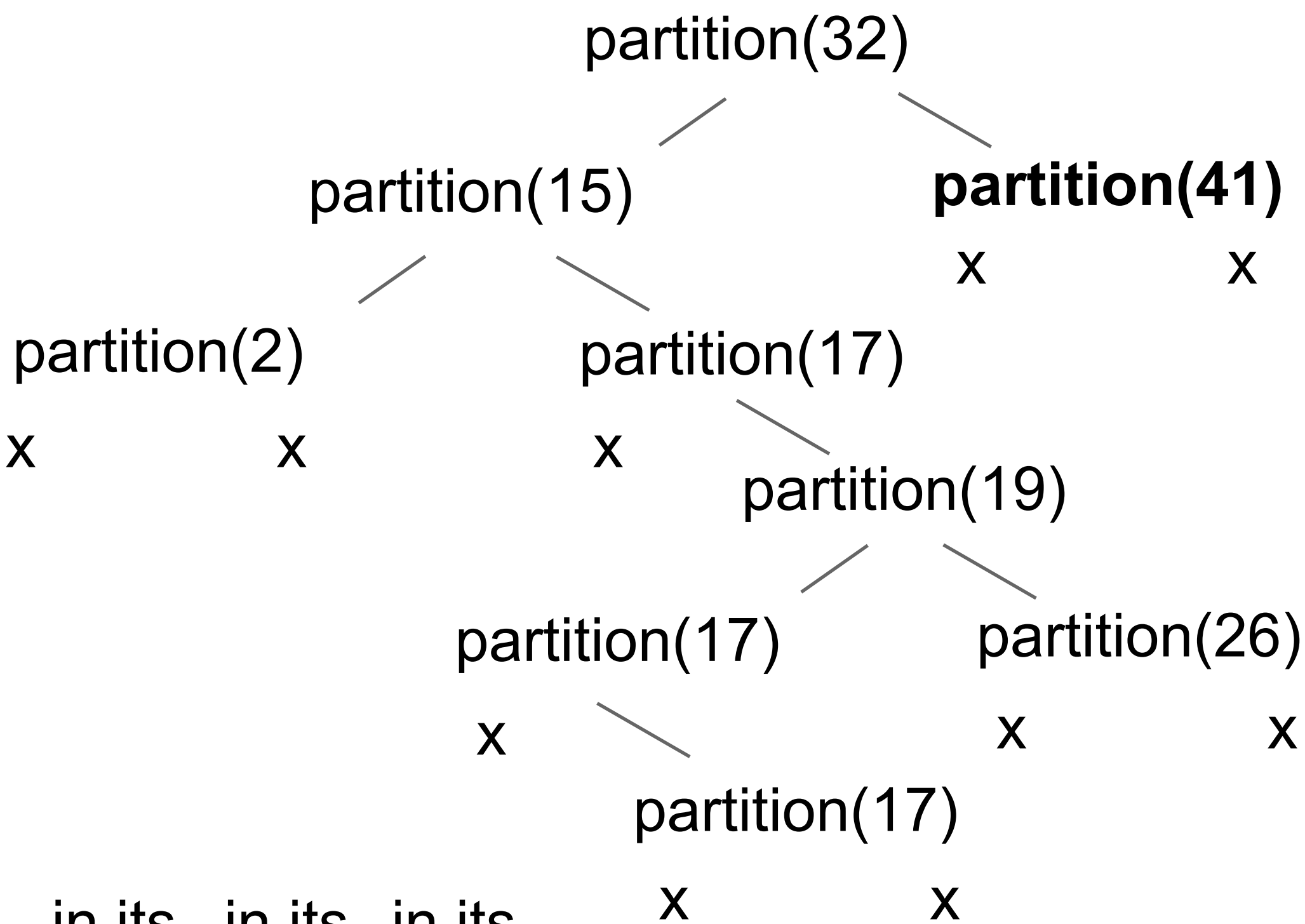
Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 17 | 17 | 19 | 26 | 32 | 41 |
|---|----|----|----|----|----|----|----|----|

# Quick Sort

Quick sorting N items:

- Partition on leftmost item (41).
- Quicksort left half.
- Quicksort right half.



in its in its in its in its in its in its in its in its  
place place place place place place place place

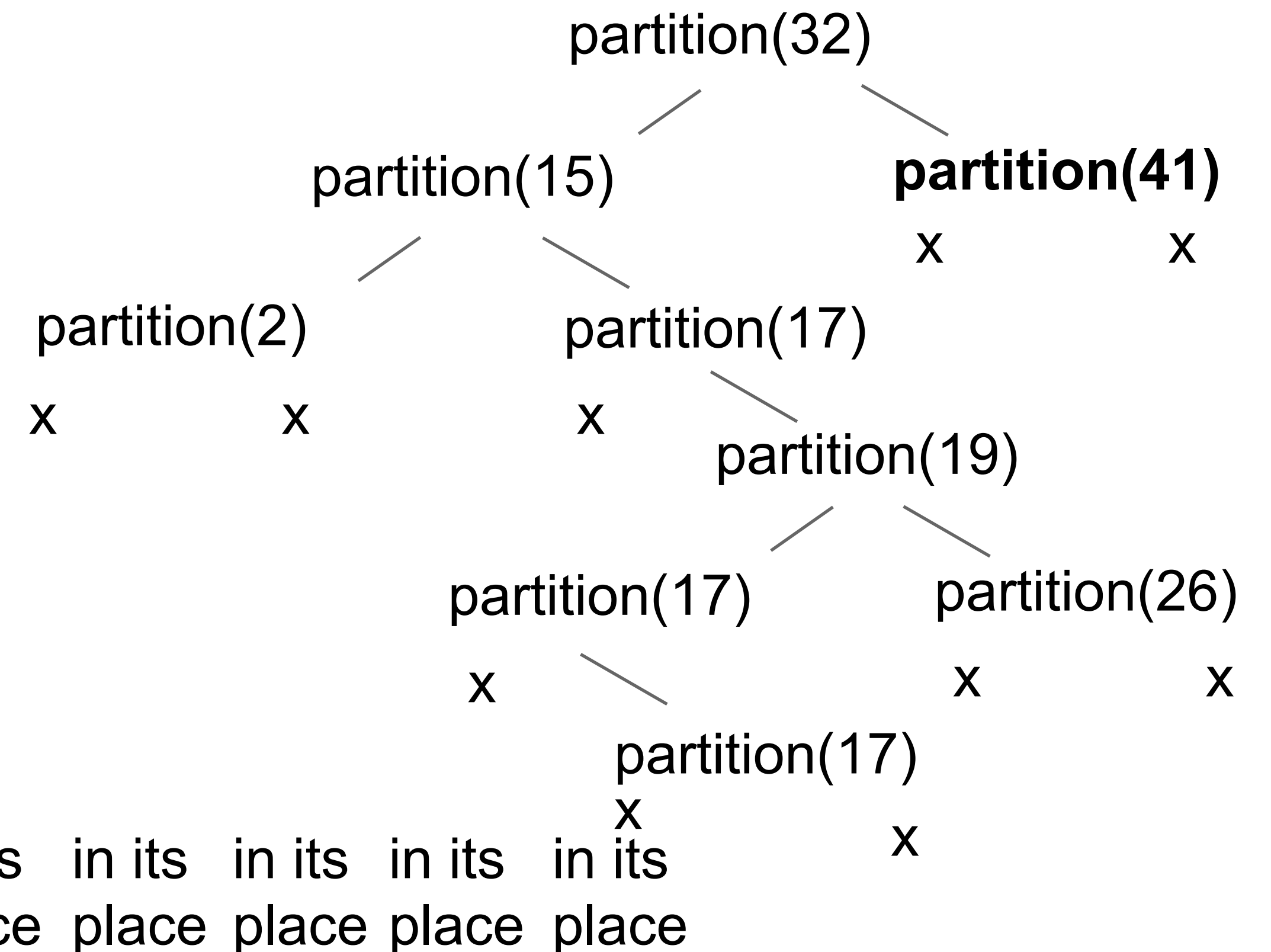
Input:

|   |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|
| 2 | 15 | 17 | 17 | 17 | 19 | 26 | 32 | 41 |
|---|----|----|----|----|----|----|----|----|

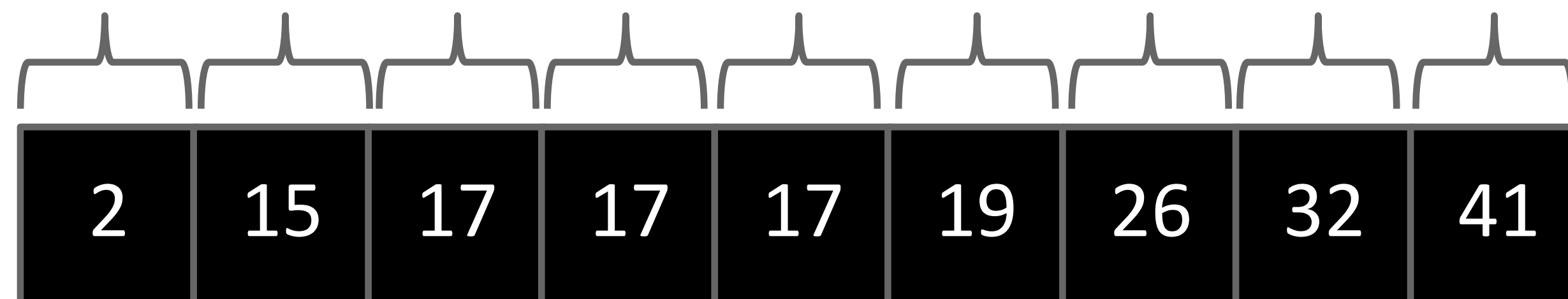
# Quick Sort

## Quick sorting N items:

- Partition on leftmost item (41) (done).
- **Quicksort left half (size zero, done).**
- **Quicksort right half (size zero, done).**



# Input:



Time complexity of quick sort?

|   | Worst         | Best          |
|---|---------------|---------------|
| A | $O(n)$        | $O(\log n)$   |
| B | $O(n^2)$      | $O(n \log n)$ |
| C | $O(n \log n)$ | $O(n)$        |
| D | $O(n \log n)$ | $O(n \log n)$ |

**class[Buzz]: quick\_sort**

# Summary

|                | Best Case     | Worst Case    | Additional Space  |
|----------------|---------------|---------------|-------------------|
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(1)$ - in place |
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(1)$ - in place |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n)$            |
| Quick Sort     | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$       |

# Heapsort

# Insert to a heap, then take out

- By inserting elements to a heap, you are generating a max- (or min-) heap
- When taking out one element at a time from the heap, you are getting the elements in descending (or ascending) order

Time complexity of heap sort?

|   | Worst         | Best          |
|---|---------------|---------------|
| A | $O(n)$        | $O(\log n)$   |
| B | $O(n^2)$      | $O(n \log n)$ |
| C | $O(n \log n)$ | $O(n)$        |
| D | $O(n \log n)$ | $O(n \log n)$ |

**class[Buzz]: heap\_sort**

# Summary: Heapsort is the winner!

|                | Best Case     | Worst Case    | Additional Space  |
|----------------|---------------|---------------|-------------------|
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(1)$ - in place |
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(1)$ - in place |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n)$            |
| Quick Sort     | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$       |
| Heap Sort      | $O(n \log n)$ | $O(n \log n)$ | $O(1)$ - in place |

# In practice what do Python libraries use?

- Interestingly, neither numpy nor the core python library use Heapsort
- Numpy uses a quicksort and falls back on heapsort (<https://towardsdatascience.com/surprising-sorting-tips-for-data-scientists-9c360776d7e>.)
- Python uses a bespoke sorting algorithm (that has since been exported) called 'Tim Sort': <https://en.wikipedia.org/wiki/Timsort>